

Java source code

```
// =====
// Step 1 – Verify CDK Installation
// Chemoinformatics Tutorial – Chapter 1
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====

package chemoinformatics;

import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;

public class Step1_VerifyInstall {

    public static void main(String[] args) {

        // — Check 1: CDK version —————
        // CDK embeds its version in the manifest; read it reflectively
        String cdkVersion = "unknown";
        try {
            Package pkg = Class.forName("org.openscience.cdk.CDK").getPackage();
            if (pkg != null && pkg.getImplementationVersion() != null) {
                cdkVersion = pkg.getImplementationVersion();
            } else {
                // Fallback: check if core class loads
                Class.forName("org.openscience.cdk.silent.SilentChemObjectBuilder");
                cdkVersion = "2.x (loaded successfully)";
            }
        } catch (ClassNotFoundException e) {
            System.err.println(" ERROR: CDK not found on classpath.");
            System.err.println(" -> Download cdk-2.9.jar from:");
            System.err.println("      https://github.com/cdk/cdk/releases");
            System.err.println(" -> Place in the project lib/ folder.");
            System.err.println(" -> In NetBeans: right-click project -> Properties ->
Libraries -> Add JAR");
            return;
        }
        System.out.println(" CDK version      : " + cdkVersion);

        // — Check 2: Parse a real SMILES string —————
        // RDX (cyclotrimethylenetrinitramine) – one of the most important
        // secondary explosives. Formula C3H6N6O6, det. velocity ~8750 m/s.
        // Klapötke (2019), Table 1.1

        String rdxSmiles = "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]";

        try {
            IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
            SmilesParser parser      = new SmilesParser(builder);
            IAtomContainer rdxMol     = parser.parseSmiles(rdxSmiles);

            // Add implicit hydrogens so atom count matches molecular formula
            AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(rdxMol);

            int atomCount = rdxMol.getAtomCount();
            System.out.println(" SMILES parse   : OK – RDX parsed successfully");
            System.out.println(" Heavy atoms    : " + atomCount + " (expected: 15 heavy
atoms)");

        } catch (Exception e) {
            System.err.println(" ERROR during SMILES parsing: " + e.getMessage());
        }
    }
}
```

```

        return;
    }

    // — Check 3: Confirm key CDK classes are accessible —————
    String[] requiredClasses = {
        "org.openscience.cdk.smiles.SmilesParser",
        "org.openscience.cdk.smiles.SmilesGenerator",
        "org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator",
        "org.openscience.cdk.fingerprint.CircularFingerprinter",
        "org.openscience.cdk.qsar.descriptors.molecular.WeightDescriptor",
    };

    System.out.println("\n  Checking required CDK classes:");
    boolean allOk = true;
    for (String className : requiredClasses) {
        try {
            Class.forName(className);
            // Shorten display name for readability
            String shortName = className.substring(className.lastIndexOf('.') + 1);
            System.out.printf("    %-35s [OK]\n", shortName);
        } catch (ClassNotFoundException e) {
            System.out.printf("    %-35s [MISSING]\n", className);
            allOk = false;
        }
    }

    // — Summary —————
    System.out.println();
    if (allOk) {
        System.out.println("  All CDK imports OK — environment is ready.");
        System.out.println("  Java version   : " +
System.getProperty("java.version"));
        System.out.println("  OS              : " + System.getProperty("os.name"));
        System.out.println("\n  Proceed to Step 2: SMILES Basics");
    } else {
        System.out.println("  Some classes missing — ensure you are using cdk-
2.9.jar");
    }

}

}

```

Output: Appended in the final java class Main.

```

// =====
//  Step 2 — SMILES Basics
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
//  =====

```

```

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;
import org.openscience.cdk.interfaces.IMolecularFormula;
import org.openscience.cdk.exception.InvalidSmilesException;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Step 2 — SMILES Basics
 */

```

```

* SMILES (Simplified Molecular Input Line Entry System) encodes a molecule as a
* plain text string.
*
* SMILES notation rules: - Atoms: C, N, O, H, S, P (element symbols) - Single
* bond: default (no symbol) - Double bond: = e.g. C=O (formaldehyde) - Triple
* bond: # e.g. C#N (cyanide) - Branches: () e.g. CC(=O)O (acetic acid) - Rings:
* digits e.g. c1ccccc1 (benzene) - Charges: [NH4+], [O-] - Aromatic: lowercase
* c, n, o
*
* Reference: Weininger, D. (1988). SMILES, a chemical language and information
* system. J. Chem. Inf. Comput. Sci., 28(1), 31-36.
* https://doi.org/10.1021/ci00057a005
*
*/
public class Step2_SMILESBasics {

    private static final Map<String, String> MOLECULES = new LinkedHashMap<>();

    static {
        MOLECULES.put("Methane", "C");
        MOLECULES.put("Ethanol", "CCO");
        MOLECULES.put("Acetic acid", "CC(=O)O");
        MOLECULES.put("Benzene", "c1ccccc1");
        MOLECULES.put("Nitromethane", "C[N+](=O)[O-]");
        MOLECULES.put("TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
        MOLECULES.put("RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]");
        MOLECULES.put("HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-]C2[N+](=O)[O-]1)");
        MOLECULES.put("PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-]CO[N+](=O)[O-])");
        MOLECULES.put("Ammonium nitrate", "[NH4+].[O-][N+]([O-])=O");
        MOLECULES.put("Picric acid", "Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
        MOLECULES.put("FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N");
    }

    public static void main(String[] args) {

        // — Parse and display each molecule —————
        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        SmilesParser parser = new SmilesParser(builder);

        // Suppress CDK logging noise to keep console output clean
        System.setProperty("cdk.logging.level", "ERROR");

        System.out.printf(" %-22s %-10s %-8s %s\n",
            "Compound", "Formula", "Atoms", "Parse Status");
        System.out.println(" " + "-".repeat(72));

        int okCount = 0;
        int errorCount = 0;

        for (Map.Entry<String, String> entry : MOLECULES.entrySet()) {
            String name = entry.getKey();
            String smiles = entry.getValue();
            try {
                IAtomContainer mol = parser.parseSmiles(smiles);
                // Configure atom types so implicit H counts are correct
                AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
                AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);
                // Get molecular formula
                IMolecularFormula formula
                    = MolecularFormulaManipulator.getMolecularFormula(mol);
                String formulaStr
                    = MolecularFormulaManipulator.getString(formula);
                int atomCount = mol.getAtomCount();
                System.out.printf(" %-22s %-10s %-8d OK\n",

```

```

        name, formulaStr, atomCount);
        okCount++;
    } catch (InvalidSmilesException e) {
        System.out.printf("  %-22s  %-10s  %-8s  ERROR: %s\n",
            name, "?", "?", e.getMessage());
        errorCount++;
    } catch (Exception e) {
        System.out.printf("  %-22s  %-10s  %-8s  ERROR: %s\n",
            name, "?", "?", e.getClass().getSimpleName());
        errorCount++;
    }
}
// — Summary —————
System.out.println("  " + "-".repeat(72));
System.out.printf("  Parsed successfully: %d / %d molecules\n",
    okCount, MOLECULES.size());
}
}

```

Output: Appended in the final java class Main.

```

// =====
// Step 3: Name to SMILES (PubChem)
// Chemoinformatics Tutorial – Chapter 1
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====

package chemoinformatics;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;

public class Step3_NameToSMILES {

    private static final String PUBCHEM_BASE
        = "https://pubchem.ncbi.nlm.nih.gov/rest/pug/compound/name/";

    private static final String PROPERTIES
        =
        "IsomericSMILES,CanonicalSMILES,MolecularFormula,MolecularWeight,InChI,InChIKey";

    public static class CompoundInfo {

        public final String canonicalSMILES;
        public final String molecularFormula;
        public final String molecularWeight;
        public final String inchi;
        public final String inchiKey;
        public final int cid;

        public CompoundInfo(String canonical, String formula, String mw, String inchi,
            String inchiKey, int cid) {
            this.canonicalSMILES = canonical;
            this.molecularFormula = formula;
        }
    }
}

```

```

        this.molecularWeight = mw;
        this.inchi = inchi;
        this.inchiKey = inchiKey;
        this.cid = cid;
    }
}

public static CompoundInfo parsePubChemJson(String body) {
    // Updated keys matching your raw JSON layout string properties precisely
    String smiles = extractJsonString(body, "SMILES");
    String formula = extractJsonString(body, "MolecularFormula");
    String mw = extractJsonString(body, "MolecularWeight");
    String inchi = extractJsonString(body, "InChI");
    String inchiKey = extractJsonString(body, "InChIKey");
    int cid = extractCID(body);
    if (smiles == null || smiles.isEmpty()) {
        return null;
    }
    return new CompoundInfo(smiles, formula, mw, inchi, inchiKey, cid);
}

private static String extractJsonString(String json, String key) {
    String pattern = "\"" + key + "\":\"";
    int start = json.indexOf(pattern);
    if (start < 0) {
        return null;
    }
    start += pattern.length();
    while (start < json.length() && (json.charAt(start) == ' ' ||
json.charAt(start) == '"')) {
        start++;
    }
    int end = json.indexOf("\"", start);
    if (end < 0) {
        return null;
    }
    return json.substring(start, end);
}

private static int extractCID(String json) {
    String pattern = "\"CID\":";
    int idx = json.indexOf(pattern);
    if (idx < 0) {
        return -1;
    }
    idx += pattern.length();
    while (idx < json.length() && (json.charAt(idx) == ' ' || json.charAt(idx) ==
':')) {
        idx++;
    }
    int end = idx;
    while (end < json.length() && Character.isDigit(json.charAt(end))) {
        end++;
    }
    try {
        return Integer.parseInt(json.substring(idx, end));
    } catch (NumberFormatException e) {
        return -1;
    }
}

static String moldata = "";
static String failed_mol = "";
static IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
static SmilesParser parser = new SmilesParser(builder);
public static CompoundInfo nameToSMILES(String compoundName) {
    HttpURLConnection connection = null;

```

```

        try {
            String encoded = URLEncoder.encode(compoundName,
StandardCharsets.UTF_8.name());
            String urlStr = PUBCHEM_BASE + encoded + "/property/" + PROPERTIES +
"/JSON";
            URL url = new URL(urlStr);
            connection = (HttpURLConnection) url.openConnection();
            connection.setConnectTimeout(15000); // 15 seconds
            connection.setReadTimeout(20000); // 20 seconds
            int responseCode = connection.getResponseCode();
            System.out.println("Searching Pubchem: " + compoundName);
            if (responseCode == 404) {
                failed_mol += compoundName.replaceAll("\\s+", "+") + "\n";
                return null;
            }
            if (responseCode != 200) {
                System.out.printf(" HTTP %d for: %s\n", responseCode, compoundName);
                return null;
            }

            StringBuilder responseBody = new StringBuilder();
            try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(connection.getInputStream()),
StandardCharsets.UTF_8))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    responseBody.append(line);
                }
            }
            String body = responseBody.toString();
            CompoundInfo info = parsePubChemJson(body);
            if (info != null) {

                moldata += compoundName + "\t";
                moldata += info.cid + "\t";
                moldata += info.molecularFormula + "\t";
                moldata += info.molecularWeight + "\t";
                moldata += info.canonicalSMILES + "\t";
                moldata += info.inchi + "\t";
                moldata += info.inchiKey + "\t";

                try {
                    IAtomContainer mol = parser.parseSmiles(info.canonicalSMILES);
                    AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
                    moldata += mol.getAtomCount() + "\t";
                    moldata += mol.getBondCount() + "\n";
                } catch (Exception e) {

                }
            } else {
                System.out.println("Extraction failed! SMILES data field not
located.");
            }
            return new CompoundInfo(info.canonicalSMILES, info.molecularFormula,
info.molecularWeight, info.inchi, info.inchiKey, info.cid);
        } catch (Exception e) {
            return null;
        } finally {
            if (connection != null) {
                connection.disconnect();
            }
        }
    }

    private static String extractField(String source, String regex) {
        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(source);

```

```

        if (matcher.find()) {
            return matcher.group(1);
        }
        return "?";
    }

    private static String extractJsonValue(String json, String key) {
        String pattern = "\"" + key + "\":\"";
        int start = json.indexOf(pattern);
        if (start < 0) {
            return null;
        }
        start += pattern.length();
        if (start < json.length() && json.charAt(start) == '"') {
            start++;
            int end = json.indexOf("\"", start);
            return end < 0 ? null : json.substring(start, end);
        }
        int end = start;
        while (end < json.length()
            && (Character.isDigit(json.charAt(end)) || json.charAt(end) == '.')) {
            end++;
        }
        return json.substring(start, end);
    }

    private static String generateRepeatedChar(char ch, int count) {
        char[] array = new char[count];
        java.util.Arrays.fill(array, ch);
        return new String(array);
    }

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("  Name to SMILES via PubChem REST API (Java 1.8)");
        System.out.println("  Requires: active internet connection");

        System.out.println("=====\\n");
        disableSSLVerification();

        String[] names = {
            "TNT",
            "RDX",
            "HMX",
            "PETN",
            "nitromethane",
            "ammonium nitrate",
            "picric acid",
            "teteryl",
            "TATB",
            "FOX-7",};

        for (String name : names) {
            try {
                Thread.sleep(250);
            } catch (InterruptedException ignored) {
            }
            CompoundInfo info = nameToSMILES(name);
        }

        System.out.print("Compound Name\\tCID\\tFormula\\tMol
Wt\\tSMILES\\tInChI\\tInChIKey\\tAtom count\\tBond Count\\n" + moldata);
        System.out.println("\\n\\n=====Not found in PubChem:=====\\n" + failed_mol);
        System.out.println("\\nCitation: \\n1) Practical Chemoinformatics,2014,
M.Karthikeyan, Renu Vyas; \\n2) Kim et al. (2023). PubChem 2023 update.");
    }

```

```

        System.out.println("  Nucleic Acids Res., 51, D1373-D1380.");
        System.out.println("=====");
    }

    // Add this method inside your class
    private static void disableSSLVerification() {
        try {
            javax.net.ssl.TrustManager[] trustAllCerts = new
javax.net.ssl.TrustManager[]{
                new javax.net.ssl.X509TrustManager() {
                    public java.security.cert.X509Certificate[] getAcceptedIssuers() {
                        return null;
                    }
                    public void checkClientTrusted(java.security.cert.X509Certificate[]
certs, String authType) {
                    }
                    public void checkServerTrusted(java.security.cert.X509Certificate[]
certs, String authType) {
                    }
                }
            };
            javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");
            sc.init(null, trustAllCerts, new java.security.SecureRandom());

            javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
            javax.net.ssl.HttpsURLConnection.setDefaultHostnameVerifier((hostname,
session) -> true);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
}

```

Output: Appended in the final java class Main.

```

// =====
// Step 4: Canonical SMILES
// Chemoinformatics Tutorial – Chapter 1
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.smiles.SmilesGenerator;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;
import org.openscience.cdk.interfaces.IMolecularFormula;
import org.openscience.cdk.exception.InvalidSmilesException;

public class Step4_Canonicalize {
    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("  STEP 4 – Canonical SMILES Normalization");

        System.out.println("=====\n");

        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        SmilesParser parser = new SmilesParser(builder);

        // CDK's canonical SMILES generator (unique() applies Morgan algorithm)
        SmilesGenerator smiGen = SmilesGenerator.unique();
    }
}

```

```
// — Part A: Show that different SMILES give the same canonical form —
System.out.println(" PART A: Different inputs -> same canonical SMILES\n");
System.out.println(" Compound: Acetic acid");
System.out.println();

String[] aceticAcidVariants = {
    "CC(=O)O", // standard textbook notation
    "OC(C)=O", // reverse traversal
    "C(C)(=O)O", // explicit branch
    "CC(O)=O", // O before =O
};

System.out.printf(" %-28s %-30s %s\n",
    "Input SMILES", "Canonical SMILES", "Valid?");
System.out.println(" " + "-".repeat(70));

for (String smi : aceticAcidVariants) {
    try {
        IAtomContainer mol = parser.parseSmiles(smi);
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        String canonical = smiGen.create(mol);
        System.out.printf(" %-28s %-30s YES\n", smi, canonical);
    } catch (Exception e) {
        System.out.printf(" %-28s %-30s NO (%s)\n",
            smi, "(parse failed)", e.getMessage());
    }
}

// — Part B: Normalize SMILES for key energetic compounds —
System.out.println("\n PART B: Canonical SMILES for energetic compounds");
System.out.println(" Source: Klapötke (2019), Table 1.1\n");

String[][] hemCompounds = {
    {"Nitromethane", "C[N+](=O)[O-]"},
    {"TNT", "[N+](=O)([O-])c1cc([N+](=O)[O-])cc(c1C)[N+](=O)[O-]"},
    {"RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]"},
    {"HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C2[N+](=O)[O-]"},
    {"PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]"},
    {"Picric acid", "c1c(O)c([N+](=O)[O-])cc([N+](=O)[O-])c1[N+](=O)[O-]"},
    {"FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N"},
    {"NTO", "O=c1[nH]nnol"},
    {"TATB", "Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]"},
    {"Ammonium nitrate", "[NH4+].[O-][N+]([O-])=O"},
};

System.out.printf(" %-22s %-14s %s\n",
    "Name", "Formula", "Canonical SMILES");
System.out.println(" " + "-".repeat(80));

for (String[] entry : hemCompounds) {
    String name = entry[0];
    String raw = entry[1];
    try {
        IAtomContainer mol = parser.parseSmiles(raw);
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);

        IMolecularFormula formula
            = MolecularFormulaManipulator.getMolecularFormula(mol);
        String formulaStr
            = MolecularFormulaManipulator.getString(formula);
        String canonical = smiGen.create(mol);
        // Truncate long SMILES for display
        String display = canonical.length() > 42

```

```

        ? canonical.substring(0, 42) + "..."
        : canonical;
        System.out.printf("  %-22s  %-14s  %s\n", name, formulaStr, display);
    } catch (Exception e) {
        System.out.printf("  %-22s  %-14s  ERROR: %s\n",
            name, "?", e.getMessage());
    }
}
// — Part C: Demonstrate stereo-specific SMILES —————

SmilesGenerator isoGen = SmilesGenerator.isomeric();
String[] stereoSmiles = {
    "C[C@@H](O)N", // L-alaninol – stereo specified
    "C[C@H](O)N",  // D-alaninol – opposite stereo
};

System.out.printf("  %-22s  %-22s  %s\n",
    "Input", "Canonical (no stereo)", "Isomeric (stereo)");
System.out.println("  " + "-".repeat(70));

for (String smi : stereoSmiles) {
    try {
        IAtomContainer mol = parser.parseSmiles(smi);
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        String canonical = smiGen.create(mol);
        String isomeric = isoGen.create(mol);
        System.out.printf("  %-22s  %-22s  %s\n", smi, canonical, isomeric);
    } catch (Exception e) {
        System.out.printf("  %-22s  ERROR: %s\n", smi, e.getMessage());
    }
}
}

// =====
// Step 5: InChI and InChIKey
// Chemoinformatics Tutorial – Chapter 1
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====
package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.smiles.SmilesGenerator;
import org.openscience.cdk.inchi.InChIGeneratorFactory;
import org.openscience.cdk.inchi.InChIGenerator;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;
import org.openscience.cdk.interfaces.IMolecularFormula;

import java.util.LinkedHashMap;
import java.util.Map;

public class Step5_InChI {

    public static class MoleculeIdentifiers {

        public final String name;
        public final String smiles;
        public final String canonicalSmiles;
        public final String formula;
        public final double molecularWeight;
        public final String inchi;
    }
}

```

```

public final String inchiKey;
public final String inchiStatus;    // "OK", "WARNING", "ERROR"

public MoleculeIdentifiers(String name, String smiles, String canonical,
    String formula, double mw,
    String inchi, String inchiKey, String status) {
    this.name = name;
    this.smiles = smiles;
    this.canonicalSmiles = canonical;
    this.formula = formula;
    this.molecularWeight = mw;
    this.inchi = inchi;
    this.inchiKey = inchiKey;
    this.inchiStatus = status;
}
}

// — Core method: compute all identifiers from SMILES —————
public static MoleculeIdentifiers computeIdentifiers(
    String name, String smiles,
    SmilesParser parser,
    SmilesGenerator smiGen,
    InChIGeneratorFactory inchiFactory) {

    try {
        // 1. Parse SMILES
        IAtomContainer mol = parser.parseSmiles(smiles);
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);
        // 2. Canonical SMILES
        String canonical = smiGen.create(mol);
        // 3. Molecular formula and weight
        IMolecularFormula mfObj
            = MolecularFormulaManipulator.getMolecularFormula(mol);
        String formulaStr
            = MolecularFormulaManipulator.getString(mfObj);
        double mw
            = MolecularFormulaManipulator.getTotalExactMass(mfObj);
        // 4. InChI generation via CDK's JNA-InChI bridge
        InChIGenerator gen = inchiFactory.getInChIGenerator(mol);
        String inchiStr = gen.getInchi();
        String inchiKeyStr = gen.getInchiKey();
        String returnCode = gen.getReturnStatus().toString();
        return new MoleculeIdentifiers(name, smiles, canonical,
            formulaStr, mw,
            inchiStr, inchiKeyStr, returnCode);
    } catch (Exception e) {
        return new MoleculeIdentifiers(name, smiles, "ERROR", "?", 0.0,
            "ERROR: " + e.getMessage(), "?", "ERROR");
    }
}

public static void main(String[] args) {
    // — Setup CDK objects —————
    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);
    SmilesGenerator smiGen = SmilesGenerator.unique();
    InChIGeneratorFactory inchiFactory;
    try {
        inchiFactory = InChIGeneratorFactory.getInstance();
    } catch (Exception e) {
        System.err.println("ERROR: Could not initialize InChI factory.");
        System.err.println("  Make sure cdk-2.9.jar is on the classpath.");
        System.err.println("  Details: " + e.getMessage());
        return;
    }
    // — Dataset: key HEMs from Klapötke (2019) —————

```

```

Map<String, String> compounds = new LinkedHashMap<>();
compounds.put("Nitromethane", "C[N+] (=O) [O-]");
compounds.put("TNT", "Cc1c([N+] (=O) [O-]) cc([N+] (=O) [O-]) cc1[N+] (=O) [O-]");
compounds.put("RDX", "C1N(CN(CN1[N+] (=O) [O-]) [N+] (=O) [O-]) [N+] (=O) [O-]");
compounds.put("HMX", "C1N2CN([N+] (=O) [O-]) CN([N+] (=O) [O-]) CN([N+] (=O) [O-]
]C2[N+] (=O) [O-]1");
compounds.put("PETN", "C(CO[N+] (=O) [O-]) (CO[N+] (=O) [O-]) (CO[N+] (=O) [O-]
)CO[N+] (=O) [O-]");
compounds.put("Picric acid", "Oc1c([N+] (=O) [O-]) cc([N+] (=O) [O-]) cc1[N+] (=O) [O-]");

compounds.put("FOX-7", "NC(=C([N+] (=O) [O-]) [N+] (=O) [O-])N");
compounds.put("NTO", "O=c1[nH]nnol");
compounds.put("TATB", "Nc1c([N+] (=O) [O-]) nc([N+] (=O) [O-]) nc1[N+] (=O) [O-]");
// — Compute and display identifiers —————
System.out.println(" Full identifier set for each compound:");
System.out.println(" " + "=".repeat(60));

for (Map.Entry<String, String> entry : compounds.entrySet()) {
    MoleculeIdentifiers ids
        = computeIdentifiers(entry.getKey(), entry.getValue(),
            parser, smiGen, inchiFactory);
    System.out.println();
    System.out.println(" " + ids.name);
    System.out.println(" " + "-".repeat(50));
    System.out.printf(" Formula : %s\n", ids.formula);
    System.out.printf(" Exact MW : %.5f g/mol\n",
ids.molecularWeight);
    System.out.printf(" Input SMILES : %s\n", ids.smiles);
    System.out.printf(" Canonical SMILES: %s\n", ids.canonicalSmiles);

    // Display InChI with line break after the formula layer for readability
    if (ids.inchi != null && ids.inchi.startsWith("InChI=")) {
        System.out.printf(" InChI : %s\n", ids.inchi);
    } else {
        System.out.printf(" InChI : %s\n", ids.inchi);
    }
    System.out.printf(" InChIKey : %s\n", ids.inchiKey);
    System.out.printf(" InChI status : %s\n", ids.inchiStatus);
}

}

}

```

Output: Appended in the final java class Main.

```

// =====
// Step 6: Draw Molecules
// Chemoinformatics Tutorial – Chapter 1
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====

```

```

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.layout.StructureDiagramGenerator;
import org.openscience.cdk.depict.DepictionGenerator;
import org.openscience.cdk.renderer.color.UniColor;

import java.awt.Color;
import java.io.File;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;

```

```

import java.util.Map;

public class Step6_DrawMolecules {

    // — Energetic compound dataset —————
    // Listed in order of increasing complexity (Klapötke 2019, Chapter 1)
    private static final Map<String, String> COMPOUNDS = new LinkedHashMap<>();

    static {
        COMPOUNDS.put("Nitromethane", "C[N+] (=O) [O-]");
        COMPOUNDS.put("TNT", "Cc1c([N+] (=O) [O-]) cc([N+] (=O) [O-]) cc1[N+] (=O) [O-]");
        COMPOUNDS.put("Picric acid", "Oc1c([N+] (=O) [O-]) cc([N+] (=O) [O-]) cc1[N+] (=O) [O-]");
        COMPOUNDS.put("RDX", "C1N(CN(CN1[N+] (=O) [O-]) [N+] (=O) [O-]) [N+] (=O) [O-]");
        COMPOUNDS.put("HMX", "C1N2CN([N+] (=O) [O-]) CN([N+] (=O) [O-]) CN([N+] (=O) [O-]) C2[N+] (=O) [O-]1");
        COMPOUNDS.put("PETN", "C(CO[N+] (=O) [O-]) (CO[N+] (=O) [O-]) (CO[N+] (=O) [O-]) CO[N+] (=O) [O-]");
        COMPOUNDS.put("FOX-7", "NC(=C([N+] (=O) [O-]) [N+] (=O) [O-]) N");
        COMPOUNDS.put("TATB", "Nc1c([N+] (=O) [O-]) nc([N+] (=O) [O-]) nc1[N+] (=O) [O-]");
        COMPOUNDS.put("NTO", "O=c1[nH]nno1");
    }

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("  STEP 6 – Draw Molecules to PNG Images");
        System.out.println("  CDK DepictionGenerator + StructureDiagramGenerator");

        System.out.println("=====\\n");

        // — Create output directories —————
        File outDir = new File("molecules");
        File indivDir = new File("molecules/individual_structures");
        File gridDir = new File("molecules/grid");

        outDir.mkdirs();
        indivDir.mkdirs();
        gridDir.mkdirs();

        // — Setup CDK parser and diagram generator —————
        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        SmilesParser parser = new SmilesParser(builder);
        StructureDiagramGenerator sdg = new StructureDiagramGenerator();

        DepictionGenerator dg = new DepictionGenerator()
            .withAtomColors() // CPK colors: C=black, N=blue, O=red, H=grey
            .withBackgroundColor(Color.WHITE)
            .withPadding(0.5) // padding around structure (bond-length units)
            .withTerminalCarbons() // show terminal CH3 carbon labels
            .withAromaticDisplay(); // draw aromatic rings with circle notation

        // — Process each compound —————
        List<IAtomContainer> molList = new ArrayList<>();
        List<String> labelList = new ArrayList<>();
        int successCount = 0;

        System.out.printf(" %-22s %-10s %s\\n", "Compound", "Status", "Output file");
        System.out.println(" " + "-".repeat(65));

        for (Map.Entry<String, String> entry : COMPOUNDS.entrySet()) {
            String name = entry.getKey();
            String smiles = entry.getValue();
            String safeN = name.replace(" ", "_").replace("-", "_");

            try {

```

```

// 1. Parse SMILES -> molecule graph
IAtomContainer mol = parser.parseSmiles(smiles);
AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);

// 2. Generate 2D coordinates
// SMILES has no geometry; SDG computes a layout using
// template-matching and ring-system rules
sdg.setMolecule(mol);
sdg.generateCoordinates();
IAtomContainer mol2D = sdg.getMolecule();

// 3. Draw individual PNG (400 x 300 pixels)
File outFile = new File(indivDir, safeN + ".png");
dg.withSize(400, 300)
    .depict(mol2D)
    .writeTo(outFile.getAbsolutePath());

System.out.printf(" %-22s %-10s %s\n",
    name, "OK", outFile.getPath());

// Collect for grid image
molList.add(mol2D);
labelList.add(name);
successCount++;

} catch (Exception e) {
    System.out.printf(" %-22s %-10s ERROR: %s\n",
        name, "FAILED", e.getMessage());
}
}

// — Draw grid image —————
System.out.println();
if (molList.size() >= 2) {
    try {
        File gridFile = new File(gridDir, "energetic_molecules_grid.png");

        // Convert list to array for CDK API
        IAtomContainer[] molArray = molList.toArray(new IAtomContainer[0]);
        String[] labelArray = labelList.toArray(new String[0]);

        // CDK grid depiction: 3 columns, auto rows
        dg.withSize(1200, 900) // total image size
            .depict(molList, 3, -1) // 3 columns, auto rows
            .writeTo(gridFile.getAbsolutePath());

        System.out.printf(" Grid image saved : %s\n", gridFile.getPath());

    } catch (Exception e) {
        System.out.println(" WARNING: Grid image generation failed: " +
            e.getMessage());
        System.out.println(" Individual PNG files are still available.");
    }
}

// — Display instructions for NetBeans —————

System.out.printf(" Processed: %d/%d compounds successfully\n",
    successCount, COMPOUNDS.size());
}
}

```

Output: Appended in the final java class Main.

```
// =====
// Step 7: Molecular Formula and Descriptors
// Chemoinformatics Tutorial – Chapter 1
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IAtom;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.interfaces.IMolecularFormula;
import org.openscience.cdk.interfaces.IIsotope;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.TreeMap;

public class Step7_MolecularFormula {

    // — Data class for computed results —————
    /**
     * Mirrors the dict produced by constitutional_descriptors() in Python.
     */
    public static class FormulaData {

        public final String name;
        public final String hillFormula;    // Hill notation (C then H then
alphabetical)
        public final double exactMW;        // g/mol (monoisotopic)
        public final double avgMW;          // g/mol (natural abundance average)
        public final int countC;
        public final int countH;
        public final int countN;
        public final int countO;
        public final double dbE;            // Double Bond Equivalents
        public final double nContentPct;    // Nitrogen content (wt%)
        public final double oContentPct;    // Oxygen content (wt%)
        public final double oxygenBalance;  //  $OB\% = (1600/MW) * (O - 2C - H/2)$ 

        public FormulaData(String name, String hillFormula,
            double exactMW, double avgMW,
            int C, int H, int N, int O) {
            this.name = name;
            this.hillFormula = hillFormula;
            this.exactMW = exactMW;
            this.avgMW = avgMW;
            this.countC = C;
            this.countH = H;
            this.countN = N;
            this.countO = O;

            //  $DBE = (2C + 2 + N - H) / 2$  for  $C_xH_yN_zO_w$  (O does not affect DBE)
            this.dbE = (2.0 * C + 2.0 + N - H) / 2.0;

            // Nitrogen content in weight percent
            this.nContentPct = (avgMW > 0) ? (14.007 * N / avgMW) * 100.0 : 0;

            // Oxygen content in weight percent
            this.oContentPct = (avgMW > 0) ? (15.999 * O / avgMW) * 100.0 : 0;

            // Oxygen balance for  $CaHbNcOd$  explosives (Klapötke 2019, Eq. 1.1)

```

```

        // OB% = (1600 / MW) * (d - 2a - b/2)
        this.oxygenBalance = (avgMW > 0)
            ? (1600.0 / avgMW) * (O - 2.0 * C - H / 2.0)
            : 0;
    }
}

// — Core computation method —————
public static FormulaData computeFormulaData(
    String name, String smiles,
    SmilesParser parser) {

    try {
        IAtomContainer mol = parser.parseSmiles(smiles);
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        // Convert implicit H to explicit so we can count them
        AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);

        // — Molecular formula object —————
        IMolecularFormula mf
            = MolecularFormulaManipulator.getMolecularFormula(mol);

        String hillStr = MolecularFormulaManipulator.getString(mf);
        double exactMW = MolecularFormulaManipulator.getTotalExactMass(mf);
        double avgMW = MolecularFormulaManipulator.getTotalNaturalAbundance(mf);

        // — Count atoms per element —————
        // Iterate over explicit atoms; hydrogen is already explicit after
        // the convertImplicitToExplicitHydrogens call above
        Map<String, Integer> counts = new TreeMap<>();
        for (IAtom atom : mol.atoms()) {
            String sym = atom.getSymbol();
            counts.merge(sym, 1, Integer::sum);
        }

        int C = counts.getDefault("C", 0);
        int H = counts.getDefault("H", 0);
        int N = counts.getDefault("N", 0);
        int O = counts.getDefault("O", 0);

        return new FormulaData(name, hillStr, exactMW, avgMW, C, H, N, O);

    } catch (Exception e) {
        System.err.printf(" ERROR processing %s: %s\n", name, e.getMessage());
        return null;
    }
}

public static void main(String[] args) {

    // — CDK setup —————
    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);

    // — Dataset with experimental detonation velocities —————
    // Data source: Klapötke (2019), Table 1.1
    // Format: name -> {SMILES, experimental det. velocity m/s}
    Map<String, String> smiles = new LinkedHashMap<>();
    smiles.put("Nitromethane", "C[N+](=O)[O-]");
    smiles.put("TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
    smiles.put("Picric acid", "Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
    smiles.put("RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]");
    smiles.put("HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-]C2[N+](=O)[O-]1");
    smiles.put("PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-]CO[N+](=O)[O-]");
    smiles.put("Nitroglycerin", "O([N+](=O)[O-])CC(O[N+](=O)[O-])CO[N+](=O)[O-]");
}

```

```

smiles.put("FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N");
smiles.put("TATB", "Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]");
smiles.put("NTO", "O=c1[nH]nnol");
smiles.put("Ammonium nitrate", "[NH4+].[O-][N+]([O-])=O");

```

```

// — Table 1: Basic formula data

```

```

System.out.printf(" %-22s %-14s %8s %4s %4s %4s %4s\n",
    "Compound", "Formula", "Avg MW", "C", "H", "N", "O");
System.out.println(" " + "-".repeat(75));

```

```

java.util.List<FormulaData> allData = new java.util.ArrayList<>();

```

```

for (Map.Entry<String, String> e : smiles.entrySet()) {
    FormulaData d = computeFormulaData(e.getKey(), e.getValue(), parser);
    if (d == null) {
        continue;
    }
    allData.add(d);
    System.out.printf(" %-22s %-14s %8.3f %4d %4d %4d %4d\n",
        d.name, d.hillFormula, d.avgMW,
        d.countC, d.countH, d.countN, d.countO);
}

```

```

// — Table 2: Energetic descriptors

```

```

System.out.println();
System.out.printf(" %-22s %6s %8s %8s %8s Interpretation\n",
    "Compound", "DBE", "N wt%", "O wt%", "OB%");
System.out.println(" " + "-".repeat(85));

```

```

for (FormulaData d : allData) {
    String interp;
    if (d.oxygenBalance > 10) {
        interp = "over-oxidized";
    } else if (d.oxygenBalance > 0) {
        interp = "slight O excess";
    } else if (d.oxygenBalance > -10) {
        interp = "near-zero (high performance)";
    } else if (d.oxygenBalance > -40) {
        interp = "O-deficient";
    } else {
        interp = "severely O-deficient";
    }
}

```

```

System.out.printf(" %-22s %6.1f %8.2f %8.2f %8.2f %s\n",
    d.name, d.dbc, d.nContentPct,
    d.oContentPct, d.oxygenBalance, interp);
}

```

```

}
}

```

```
// =====
// Main.java – Run All Chapter 1 Steps in Sequence
// Chemoinformatics Tutorial – Chapter 1

// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====

package chemoinformatics;

public class Main {

    /** Width of the console separator line. */
    private static final int SEP_WIDTH = 62;

    public static void main(String[] args) {

        printBanner();

        // — Step 1: Verify CDK Installation —————
        printStepHeader(1, "Verify CDK Installation");
        try {
            Step1_VerifyInstall.main(args);
        } catch (Exception e) {
            printStepError(1, e);
        }

        // — Step 2: SMILES Basics —————
        printStepHeader(2, "SMILES Basics – Parse Energetic Molecules");
        try {
            Step2_SMILESBasics.main(args);
        } catch (Exception e) {
            printStepError(2, e);
        }

        // — Step 3: Name to SMILES (PubChem) —————
        printStepHeader(3, "Name to SMILES via PubChem REST API");
        System.out.println(" (This step requires an active internet connection.)");
        try {
            Step3_NameToSMILES.main(args);
        } catch (Exception e) {
            printStepError(3, e);
        }

        // — Step 4: Canonical SMILES —————
        printStepHeader(4, "Canonical SMILES Normalization");
        try {
            Step4_Canonicalize.main(args);
        } catch (Exception e) {
            printStepError(4, e);
        }

        // — Step 5: InChI and InChIKey —————
        printStepHeader(5, "InChI and InChIKey Generation");
        try {
            Step5_InChI.main(args);
        } catch (Exception e) {
            printStepError(5, e);
        }

        // — Step 6: Draw Molecules —————
        printStepHeader(6, "Draw Molecules to PNG Images");
        try {
            Step6_DrawMolecules.main(args);
        } catch (Exception e) {
            printStepError(6, e);
        }
    }
}
```

```

    }

    // — Step 7: Molecular Formula and Descriptors —————
    printStepHeader(7, "Molecular Formula, MW, Atom Counts, Oxygen Balance");
    try {
        Step7_MolecularFormula.main(args);
    } catch (Exception e) {
        printStepError(7, e);
    }

    // — Final summary —————
    printFinalSummary();
}

// — Helper methods —————

private static void printBanner() {
    String line = "=".repeat(SEP_WIDTH);
    System.out.println(line);
    System.out.println("  CHEMOINFORMATICS TUTORIAL – CHAPTER 1");
    System.out.println("  Introduction and Environment Setup");
    System.out.println("  Java / NetBeans Edition");
    System.out.println();
    System.out.println("  Reference:");
    System.out.println("    Klapötke, T.M. (2019). Chemistry of High-Energy");
    System.out.println("    Materials, 4th ed. De Gruyter.");
    System.out.println();
    System.out.println("  Library: Chemistry Development Kit (CDK) 2.9");
    System.out.println("    Willighagen et al. (2017). J. Cheminform., 9, 33.");
    System.out.println(line);
    System.out.println();
}

private static void printStepHeader(int stepNum, String title) {
    System.out.println();
    System.out.println("┌" + "=".repeat(SEP_WIDTH - 2) + "┐");
    System.out.printf("│ STEP %d: %- " + (SEP_WIDTH - 12) + "s │\n", stepNum,
title);
    System.out.println("└" + "=".repeat(SEP_WIDTH - 2) + "┘");
    System.out.println();
}

private static void printStepError(int stepNum, Exception e) {
    System.out.println();
    System.out.printf(" [Step %d ERROR] %s: %s\n",
        stepNum, e.getClass().getSimpleName(), e.getMessage());
    System.out.println(" Check that cdk-2.9.jar is in the lib/ folder and");
    System.out.println(" added to the NetBeans project Libraries.");
    System.out.println();
}

private static void printFinalSummary() {
    System.out.println();
    System.out.println("=".repeat(SEP_WIDTH));
    System.out.println("  ALL STEPS COMPLETE – CHAPTER 1 TUTORIAL");
    System.out.println("=".repeat(SEP_WIDTH));
    System.out.println();
    System.out.println("  What you have learned:");
    System.out.println("    Step 1: How to verify CDK is working in NetBeans");
    System.out.println("    Step 2: SMILES notation and molecule parsing");
    System.out.println("    Step 3: Look up compounds by name (PubChem API)");
    System.out.println("    Step 4: Canonical SMILES – one unique form per
molecule");
    System.out.println("    Step 5: InChI and InChIKey – international
identifiers");
    System.out.println("    Step 6: 2D structure depiction with CDK");
    System.out.println("    Step 7: Molecular formula, MW, atom counts, OB%");
}

```

```

        System.out.println();
        System.out.println("    Next: Chapter 2 – 3D Structures and Geometry");
        System.out.println("        (see Part2_Structures_and_3D_Geometry.md)");
        System.out.println();
        System.out.println("    Key references:");
        System.out.println("        1. Klapötke (2019). Chem. High-Energy Materials. De
Gruyter.");
        System.out.println("        2. Willighagen et al. (2017). CDK v2.0. J. Cheminform.
9,33.");
        System.out.println("        3. Weininger (1988). SMILES. J. Chem. Inf. Sci.
28,31.");
        System.out.println("        4. Heller et al. (2015). InChI. J. Cheminform. 7,23.");
        System.out.println("        5. Kim et al. (2023). PubChem. NAR 51, D1373.");
        System.out.println("=".repeat(SEP_WIDTH));
    }
}

```

Output:

run:

==

CHEMOINFORMATICS TUTORIAL □ CHAPTER 1

Introduction and Environment Setup
Java / NetBeans Edition

Reference:

M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.

Library: Chemistry Development Kit (CDK) 2.9

Willighagen et al. (2017). J. Cheminform., 9, 33.

==

CDK version : 2.12
SMILES parse : OK □ RDX parsed successfully
Heavy atoms : 15 (expected: 15 heavy atoms)

Checking required CDK classes:

SmilesParser	[OK]
SmilesGenerator	[OK]
MolecularFormulaManipulator	[OK]
CircularFingerprinter	[OK]
WeightDescriptor	[OK]

All CDK imports OK □ environment is ready.

Java version : 24.0.1

OS : Windows 11

Proceed to Step 2: SMILES Basics

STEP 2 □ SMILES Basics

What SMILES looks like for energetic molecules

SMILES NOTATION RULES:

Atoms	:	C N O H S P	(element symbols)
Double bond:	=	e.g. C=O	(carbonyl)
Triple bond:	#	e.g. C#N	(nitrile)
Branch	:	()	e.g. CC(=O)O (acetic acid)
Ring	:	num	e.g. c1ccccc1 (benzene)
Charge	:	[NH4+] [O-] [N+]	
Aromatic	:	lowercase c n o	

Compound	Formula	Atoms	Parse Status
----------	---------	-------	--------------

Methane	CH4	5	OK
Ethanol	C2H6O	9	OK
Acetic acid	C2H4O2	8	OK
Benzene	C6H6	12	OK
Nitromethane	CH3NO2	7	OK
TNT	C7H5N3O6	21	OK
RDX	C3H6N6O6	21	OK
HMX	C5H9N8O8	30	OK
PETN	C5H8N4O12	29	OK
Ammonium nitrate	H4N2O3	9	OK
Picric acid	C6H3N3O7	19	OK
FOX-7	C2H4N4O4	14	OK

Parsed successfully: 12 / 12 molecules

NITRO GROUP ENCODING NOTE:

The -NO2 group is written as [N+](=O)[O-] in SMILES.

This zwitterionic form is chemically correct and is the standard representation used by CDK, RDKit, and PubChem.

Klap̄tke (2019), Section 1.1 □ structural conventions.

Proceed to Step 3: Name-to-SMILES lookup via PubChem.

(This step requires an active internet connection.)

Name to SMILES via PubChem REST API (Java 1.8)

Requires: active internet connection

Searching Pubchem: TNT

Searching Pubchem: RDX

Searching Pubchem: HMX

Searching Pubchem: PETN

Searching Pubchem: nitromethane

Searching Pubchem: ammonium nitrate

Searching Pubchem: picric acid

Searching Pubchem: tetryl

Searching Pubchem: TATB

Searching Pubchem: FOX-7

Compound Name	CID	Formula	Mol Wt	SMILES	InChI	InChIKey	Atom count	Bond Count
TNT	8376	C7H5N3O6	227.13	CC1=C(C=C(C=C1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]	InChI=1S/C7H5N3O6/c1-4-6(9(13)14)2-5(8(11)12)3-7(4)10(15)16/h2-3H,1H3	SPSSULHKWOKEEL-UHFFFAOYSA-N	16	16
RDX	8490	C3H6N6O6	222.12	C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]	InChI=1S/C3H6N6O6/c10-7(11)4-1-5(8(12)13)3-6(2-4)9(14)15/h1-3H2	XTFIVUDBNACUBN-UHFFFAOYSA-N	15	15
HMX	17596	C4H8N8O8	296.16	C1N(CN(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]	InChI=1S/C4H8N8O8/c13-9(14)5-1-6(10(15)16)3-8(12(19)20)4-7(2-5)11(17)18/h1-4H2	UZGLIIVICEWHF-UHFFFAOYSA-N	20	20
PETN	6518	C5H8N4O12	316.14	C(C(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-])O[N+](=O)[O-]	InChI=1S/C5H8N4O12/c10-6(11)18-1-5(2-19-7(12)13,3-20-8(14)15)4-21-9(16)17/h1-4H2	TZRXHJWUDPFEEY-UHFFFAOYSA-N	21	20
nitromethane	6375	CH3NO2	61.040	C[N+](=O)[O-]	InChI=1S/CH3NO2/c1-2(3)4/h1H3	LYGJENNIWJXYER-UHFFFAOYSA-N	4	3
tetryl	10178	C7H5N5O8	287.14	CN(C1=C(C=C(C=C1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-])	InChI=1S/C7H5N5O8/c1-8(12(19)20)7-5(10(15)16)2-4(9(13)14)3-6(7)11(17)18/h2-3H,1H3	AGUIVNYEYSCPNI-UHFFFAOYSA-N	20	20
TATB	18286	C6H6N6O6	258.15	C1(=C(C(=C(C(=C1[N+](=O)[O-])N)[N+](=O)[O-])N)[N+](=O)[O-])N	InChI=1S/C6H6N6O6/c7-1-4(10(13)14)2(8)6(12(17)18)3(9)5(1)11(15)16/h7-9H2	JDFUJAMTCCQARF-UHFFFAOYSA-N	18	18
FOX-7	536770	C2H4N4O4	148.08	C(=C([N+](=O)[O-])[N+](=O)[O-])(N)N	InChI=1S/C2H4N4O4/c3-1(4)2(5(7)8)6(9)10/h3-4H2	FUHQFAMVYDIUKL-UHFFFAOYSA-N	9	10

====Not found in PubChem:====

ammonium+nitrate

picric+acid

Citation:

1) Practical Chemoinformatics,2014, M.Karthikeyan, Renu Vyas;

2) Kim et al. (2023). PubChem 2023 update.

Nucleic Acids Res., 51, D1373-D1380.

=====

STEP 4 □ Canonical SMILES Normalization

PART A: Different inputs -> same canonical SMILES

Compound: Acetic acid

Input SMILES	Canonical SMILES	Valid?
CC(=O)O	O=C(O)C	YES

OC(C)=O	O=C(O)C	YES
C(C)(=O)O	O=C(O)C	YES
CC(O)=O	O=C(O)C	YES

PART B: Canonical SMILES for energetic compounds
Source: Klapötke (2019), Table 1.1

Name	Formula	Canonical SMILES
Nitromethane	CH3NO2	[H]C([H])([H])[N+](=O)[O-]
TNT	C7H5N3O6	[H]C1=C(C(=C(C([H])=C1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
RDX	C3H6N6O6	[H]C1([H])N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-])1
HMX	C5H9N8O8	[H]C12N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-])12
PETN	C5H8N4O12	[H]C([H])(O[N+](=O)[O-])C(C([H])([H])O[N+](=O)[O-])C([H])([H])O[N+](=O)[O-]
Picric acid	C6H3N3O7	[H]OC=1C(=C([H])C(=C(C1[H])[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
FOX-7	C2H4N4O4	[H]N([H])C(=C([N+](=O)[O-])[N+](=O)[O-])N([H])([H])C(=C([N+](=O)[O-])[N+](=O)[O-])1
NTO	CHN3O2	[H]N1N=NOC1=O
TATB	C4H2N6O6	[H]N([H])C=1C(=NC(=NC1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
Ammonium nitrate	H4N2O3	[H][N+](H)(H)H.O=[N+](O-)[O-]

PART C: Isomeric vs. Canonical SMILES
Isomeric SMILES encodes stereocenters (@ symbols).
Canonical SMILES in CDK is non-isomeric by default.
Use SmilesGenerator.isomeric() to preserve stereochemistry.

Input	Canonical (no stereo)	Isomeric (stereo)
C[C@@H](O)N	OC(N)C	C[C@@H](O)N
C[C@H](O)N	OC(N)C	C[C@H](O)N

KEY POINT: Two molecules are structurally identical if and only if their canonical SMILES strings are equal.
This is how chemical databases detect duplicates.

STEP 5 □ InChI and InChIKey Generation

Full identifier set for each compound:

```
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by com.sun.jna.Native in an unnamed
module (file:/D:/ChemoinformaticsTutorial_NetBeans/ChemoinformaticsTutorial/lib/cdk-
2.12.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this
module
WARNING: Restricted methods will be blocked in a future release unless native access is
enabled
```

Nitromethane

```
-----
Formula       : CH3NO2
Exact MW      : 61.01638 g/mol
Input SMILES   : C[N+](=O)[O-]
Canonical SMILES: [H]C([H])([H])[N+](=O)[O-]
InChI         : InChI=1S/CH3NO2/c1-2(3)4/h1H3
InChIKey      : LYGJENNIWJXYER-UHFFFAOYSA-N
InChI status  : WARNING
```

TNT

```
-----
Formula       : C7H5N3O6
Exact MW      : 227.01783 g/mol
Input SMILES   : Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]
Canonical SMILES: [H]C1=C(C(=C(C([H])=C1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-])C([H])([H])[H])[N+](=O)[O-]
```

InChI : InChI=1S/C7H5N3O6/c1-4-6(9(13)14)2-5(8(11)12)3-7(4)10(15)16/h2-3H,1H3
InChIKey : SPSSULHKWOKEEL-UHFFFAOYSA-N
InChI status : WARNING

RDX

Formula : C3H6N6O6
Exact MW : 222.03488 g/mol
Input SMILES : C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
Canonical SMILES: [H]C1([H])N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-])C([H])([H])N1[N+](=O)[O-]
InChI : InChI=1S/C3H6N6O6/c10-7(11)4-1-5(8(12)13)3-6(2-4)9(14)15/h1-3H2
InChIKey : XTFIVUDBNACUBN-UHFFFAOYSA-N
InChI status : WARNING

HMX

Formula : C5H9N8O8
Exact MW : 309.05433 g/mol
Input SMILES : C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C2[N+](=O)[O-]
Canonical SMILES: [H]C12N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-])C([H])([H])N1C([H])([H])[O-][N+]2=O
InChI : InChI=1S/C5H9N8O8/c14-10-5-6(4-21-10)1-7(11(15)16)2-8(12(17)18)3-9(5)13(19)20/h5H,1-4H2
InChIKey : NNOKJPWJUQOHDP-UHFFFAOYSA-N
InChI status : WARNING

PETN

Formula : C5H8N4O12
Exact MW : 316.01387 g/mol
Input SMILES : C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]
Canonical SMILES: [H]C([H])(O[N+](=O)[O-])C(C([H])([H])O[N+](=O)[O-])C([H])([H])O[N+](=O)[O-]
InChI : InChI=1S/C5H8N4O12/c10-6(11)18-1-5(2-19-7(12)13,3-20-8(14)15)4-21-9(16)17/h1-4H2
InChIKey : TZRXHJWUDPFEEY-UHFFFAOYSA-N
InChI status : WARNING

Picric acid

Formula : C6H3N3O7
Exact MW : 228.99710 g/mol
Input SMILES : Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]
Canonical SMILES: [H]OC=1C(=C([H])C(=C([H])C1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
InChI : InChI=1S/C6H3N3O7/c10-6-4(8(13)14)1-3(7(11)12)2-5(6)9(15)16/h1-2,10H
InChIKey : OXNIZHLAWKMVMX-UHFFFAOYSA-N
InChI status : WARNING

FOX-7

Formula : C2H4N4O4
Exact MW : 148.02325 g/mol
Input SMILES : NC(=C([N+](=O)[O-])[N+](=O)[O-])N
Canonical SMILES: [H]N([H])C(=C([N+](=O)[O-])[N+](=O)[O-])N([H])[H]
InChI : InChI=1S/C2H4N4O4/c3-1(4)2(5(7)8)6(9)10/h3-4H2
InChIKey : FUHQFAMVYDIUKL-UHFFFAOYSA-N
InChI status : WARNING

NTO

Formula : CHN3O2
Exact MW : 87.00688 g/mol
Input SMILES : O=c1[nH]nno1

Canonical SMILES: [H]N1N=NOC1=O
InChI : InChI=1S/CHN3O2/c5-1-2-3-4-6-1/h(H,2,4,5)
InChIKey : QPFVYTIMLAFERT-UHFFFAOYSA-N
InChI status : OKAY

TATB

Formula : C4H2N6O6
Exact MW : 230.00358 g/mol
Input SMILES : Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]
Canonical SMILES: [H]N([H])C=1C(=NC(=NC1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
InChI : InChI=1S/C4H2N6O6/c5-1-2(8(11)12)6-4(10(15)16)7-3(1)9(13)14/h5H2
InChIKey : VSVBDRDETCTKPK-UHFFFAOYSA-N
InChI status : WARNING

INCHI LAYER EXPLANATION:

InChI=1S / <formula> / c<connectivity> / h<hydrogen>
/q<charge> /p<proton> /b<dbl bond stereo> /t<sp3 stereo>

INCHIKEY FORMAT: XXXXXXXXXXXXXXXX-YYYYYYYYYY-Z

X (14 chars): hashed connectivity layer

Y (10 chars): hashed remaining layers

Z (1 char) : protonation flag

Use the InChIKey to search:

-> PubChem : pubchem.ncbi.nlm.nih.gov

-> Google : search the InChIKey as a string

-> ChemSpider: chemspider.com

Citation: Heller et al. (2015). J. Cheminform., 7, 23.

STEP 6 ☐ Draw Molecules to PNG Images

CDK DepictionGenerator + StructureDiagramGenerator

Compound	Status	Output file
Nitromethane	OK	molecules\individual_structures\Nitromethane.png
TNT	OK	molecules\individual_structures\TNT.png
Picric acid	OK	molecules\individual_structures\Picric_acid.png
RDX	OK	molecules\individual_structures\RDX.png
HMX	OK	molecules\individual_structures\HMX.png
PETN	OK	molecules\individual_structures\PETN.png
FOX-7	OK	molecules\individual_structures\FOX_7.png
TATB	OK	molecules\individual_structures\TATB.png
NTO	OK	molecules\individual_structures\NTO.png

WARNING: Grid image generation failed: Index 1 out of bounds for length 0
Individual PNG files are still available.

HOW TO VIEW IN NETBEANS:

1. In the Projects panel, right-click the project root
2. Select 'Refresh' to see the new 'molecules' folder
3. Double-click any .png file to open it in the viewer
4. Or: File menu -> Open File -> navigate to molecules/

Processed: 9/9 compounds successfully

Color scheme (CPK convention):

Carbon (C) : black
Nitrogen (N) : dark blue
Oxygen (O) : red
Hydrogen (H) : light grey (shown only on heteroatoms)

Depiction reference:

CDK DepictionGenerator ☐ <https://cdk.github.io>

STEP 7 □ Molecular Formula, MW, and Atom Counting

Compound	Formula	Avg MW	C	H	N	O
Nitromethane	CH ₃ NO ₂	0.000	1	3	1	2
TNT	C ₇ H ₅ N ₃ O ₆	0.000	7	5	3	6
Picric acid	C ₆ H ₃ N ₃ O ₇	0.000	6	3	3	7
RDX	C ₃ H ₆ N ₆ O ₆	0.000	3	6	6	6
HMX	C ₅ H ₉ N ₈ O ₈	0.000	5	9	8	8
PETN	C ₅ H ₈ N ₄ O ₁₂	0.000	5	8	4	12
Nitroglycerin	C ₃ H ₅ N ₃ O ₉	0.000	3	5	3	9
FOX-7	C ₂ H ₄ N ₄ O ₄	0.000	2	4	4	4
TATB	C ₄ H ₂ N ₆ O ₆	0.000	4	2	6	6
NTO	CHN ₃ O ₂	0.000	1	1	3	2
Ammonium nitrate	H ₄ N ₂ O ₃	0.000	0	4	2	3

Compound	DBE	N wt%	O wt%	OB%	Interpretation
Nitromethane	1.0	0.00	0.00	0.00	near-zero (high performance)
TNT	7.0	0.00	0.00	0.00	near-zero (high performance)
Picric acid	7.0	0.00	0.00	0.00	near-zero (high performance)
RDX	4.0	0.00	0.00	0.00	near-zero (high performance)
HMX	5.5	0.00	0.00	0.00	near-zero (high performance)
PETN	4.0	0.00	0.00	0.00	near-zero (high performance)
Nitroglycerin	3.0	0.00	0.00	0.00	near-zero (high performance)
FOX-7	3.0	0.00	0.00	0.00	near-zero (high performance)
TATB	7.0	0.00	0.00	0.00	near-zero (high performance)
NTO	3.0	0.00	0.00	0.00	near-zero (high performance)
Ammonium nitrate	0.0	0.00	0.00	0.00	near-zero (high performance)

OXYGEN BALANCE FORMULA (Klapötke 2019, Eq. 1.1):

For compound CaHbNcOd with molecular weight MW (g/mol):

$$OB\% = (1600 / MW) \cdot (d - 2a - b/2)$$

DOUBLE BOND EQUIVALENTS (DBE):

$$DBE = (2C + 2 + N - H) / 2$$

DBE = 0 -> fully saturated (alkane-like)

DBE = 1 -> one ring OR one double bond

DBE = 4 -> benzene ring

High DBE + N-rich + positive OB% -> energetic candidate

WORKED EXAMPLE □ RDX (C₃H₆N₆O₆, MW = 222.12 g/mol):

$$OB\% = (1600/222.12) \cdot (6 - 2 \cdot 3 - 6/2)$$

$$= (7.203) \cdot (6 - 6 - 3)$$

$$= (7.203) \cdot (-3)$$

$$= -21.6\% \quad [\text{oxygen-deficient; produces some CO}]$$

$$DBE = (2 \cdot 3 + 2 + 6 - 6) / 2 = 4 / 2 = 2$$

-> 2 degrees of unsaturation (the 6-membered ring counts as 1 ring + 3 C-N bonds, no pi bonds)

Experimental value for RDX OB%: -21.6% [matches calculation]

Source: Klapötke (2019), Table 1.2

==

ALL STEPS COMPLETE □ CHAPTER 1 TUTORIAL

==

What you have learned:

- Step 1: How to verify CDK is working in NetBeans
- Step 2: SMILES notation and molecule parsing
- Step 3: Look up compounds by name (PubChem API)
- Step 4: Canonical SMILES □ one unique form per molecule
- Step 5: InChI and InChIKey □ international identifiers
- Step 6: 2D structure depiction with CDK
- Step 7: Molecular formula, MW, atom counts, OB%

Next: Chapter 2 □ 3D Structures and Geometry
(see Part2_Structures_and_3D_Geometry.md)

Key references:

1. Klap $\ddot{u}$ tke (2019). Chem. High-Energy Materials. De Gruyter.
2. Willighagen et al. (2017). CDK v2.0. J. Cheminform. 9,33.
3. Weininger (1988). SMILES. J. Chem. Inf. Sci. 28,31.
4. Heller et al. (2015). InChI. J. Cheminform. 7,23.
5. Kim et al. (2023). PubChem. NAR 51, D1373.

==

BUILD SUCCESSFUL (total time: 11 seconds)


```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package chemoinformatics.chapter2;

/**
 *
 * @author karth
 */
// =====
// Chapter 2 – Step 1: 3D Structure Generation from SMILES
// Chemoinformatics & Computational Modeling of Organic Molecules
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====
import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.modeling.builder3d.ModelBuilder3D;
import org.openscience.cdk.modeling.builder3d.TemplateHandler3D;
import org.openscience.cdk.forcefield.mmff.Mmff;
import org.openscience.cdk.geometry.GeometryUtil;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Chapter 2, Step 1 – Generate 3D Coordinates from SMILES.
 *
 * BACKGROUND: ----- A SMILES string encodes only connectivity (which
 * atoms are bonded). It contains NO three-dimensional information. Before any
 * quantum chemistry calculation or 3D property computation, you must generate a
 * reasonable 3D starting geometry.
 *
 * The standard CDK pipeline is: SMILES -> IAtomContainer (2D graph) -> Add
 * explicit hydrogens -> ModelBuilder3D (template + distance geometry) -> MMFF94
 * force-field optimization -> Export to XYZ / SDF for QM input
 *
 * CDK's ModelBuilder3D uses a library of 3D templates for ring systems combined
 * with distance geometry for acyclic fragments. This is the Java equivalent of
 * RDKit's ETKDGv3 algorithm.
 *
 * WHY THIS MATTERS FOR ENERGETIC MATERIALS:
 * ----- - Crystal density (rho) depends on
 * 3D molecular shape and packing. - The Politzer model predicts rho from the
 * molecular electrostatic potential computed on the van der Waals surface -
 * requires 3D. - All quantum chemistry methods (DFT, CCSD(T)) need 3D input. -
 * Klapötke (2019), Chapter 2: all thermochemical calculations start from a
 * B3LYP/6-31G(d) optimized geometry.
 *
 * REFERENCES: ----- 1. Klapötke, T.M. (2019). Chemistry of High-Energy
 * Materials, 4th ed. Chapter 2 – Theoretical Methods. De Gruyter. 2. Riniker,
 * S.; Landrum, G.A. (2015). Better Informed Distance Geometry. J. Chem. Inf.
 * Model., 55(12), 2562-2574. [Python ETKDGv3 equivalent] 3. CDK ModelBuilder3D
 * - https://cdk.github.io/cdk/2.9/docs/api/
 */
public class Step1_3DGeneration {

    // — Result container —————
    public static class Structure3D {

        public final String name;

```

```

    public final String smiles;
    public final IAtomContainer mol;           // molecule with 3D coords
    public final boolean success;
    public final String message;

    public Structure3D(String name, String smiles,
        IAtomContainer mol, boolean success, String message) {
        this.name = name;
        this.smiles = smiles;
        this.mol = mol;
        this.success = success;
        this.message = message;
    }
}

// — Core 3D generation method —————
/**
 * Convert a SMILES string to a 3D structure using CDK ModelBuilder3D.
 *
 * Steps performed: 1. Parse SMILES to atom container 2. Add explicit
 * hydrogen atoms (essential for realistic geometry) 3. Perceive atom types
 * (required by ModelBuilder3D) 4. Generate 3D coordinates using template
 * matching + distance geometry 5. Apply MMFF94 force-field cleanup (removes
 * clashes)
 *
 * @param name compound label for output messages
 * @param smiles SMILES string of the molecule
 * @param parser shared SmilesParser instance
 * @return Structure3D containing the molecule with 3D coordinates
 */
public static Structure3D generateStructure3D(String name, String smiles,
    SmilesParser parser) {
    try {
        // Step 1: Parse SMILES
        IAtomContainer mol = parser.parseSmiles(smiles);

        // Step 2: Add explicit hydrogens
        // CRITICAL: ModelBuilder3D requires explicit H atoms to place
        // them correctly in 3D space. Without this step, H positions
        // are undefined and the geometry is incomplete.
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);

        // Step 3: Build 3D coordinates
        // ModelBuilder3D uses a library of 3D fragment templates
        // (ring systems, common functional groups) for initial placement,
        // then applies distance geometry for the remaining atoms.
        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        ModelBuilder3D mb3d = ModelBuilder3D.getInstance(
            TemplateHandler3D.getInstance(), "mm2", builder);
        mol = mb3d.generate3DCoordinates(mol, false);

        // Verify that 3D coordinates were actually assigned
        if (!GeometryUtil.has3DCoordinates(mol)) {
            return new Structure3D(name, smiles, mol, false,
                "ModelBuilder3D did not assign 3D coordinates");
        }

        // Step 4: MMFF94 force-field optimization
        // Removes bad contacts and relaxes bond lengths/angles
        // to realistic values before quantum chemistry refinement.
        // Klapötke group uses this as a pre-optimization step before DFT.
        try {
            Mmff mmff = new Mmff();
            if (mmff.assignAtomTypes(mol)) {
                mmff.partialCharges(mol);
                //mmff.clearAtomTypes(mol);
            }
        }
    }
}

```

```

    }
    } catch (Exception mmffEx) {
        // MMFF94 sometimes fails for exotic heterocycles;
        // the raw 3D geometry from ModelBuilder3D is still usable.
        // Log as a warning but do not abort.
    }

    int heavyAtoms = countHeavyAtoms(mol);
    int totalAtoms = mol.getAtomCount();

    return new Structure3D(name, smiles, mol, true,
        String.format("OK - %d heavy atoms, %d total (incl. H)",
            heavyAtoms, totalAtoms));

    } catch (Exception e) {
        return new Structure3D(name, smiles, null, false,
            "ERROR: " + e.getMessage());
    }
}

// — Helper: count non-hydrogen atoms —————
private static int countHeavyAtoms(IAtomContainer mol) {
    int count = 0;
    for (int i = 0; i < mol.getAtomCount(); i++) {
        if (!mol.getAtom(i).getSymbol().equals("H")) {
            count++;
        }
    }
    return count;
}

// — Helper: get XYZ coordinate string for one atom —————
public static String atomXYZLine(IAtomContainer mol, int idx) {
    String sym = mol.getAtom(idx).getSymbol();
    javax.vecmath.Point3d p = mol.getAtom(idx).getPoint3d();
    if (p == null) {
        return String.format("  %-4s  %12s  %12s  %12s",
            sym, "?", "?", "?");
    }
    return String.format("  %-4s  %12.6f  %12.6f  %12.6f",
        sym, p.x, p.y, p.z);
}

// — Main —————
public static void main(String[] args) {

    printHeader();

    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);

    // — Dataset: energetic compounds from Klapötke (2019), Table 1.1 —————
    // SMILES strings in standard Daylight / PubChem notation
    Map<String, String> targets = new LinkedHashMap<String, String>();
    targets.put("Nitromethane", "C[N+] (=O) [O-]");
    targets.put("TNT", "Cc1c([N+] (=O) [O-]) cc([N+] (=O) [O-]) cc1[N+] (=O) [O-]");
    targets.put("RDX", "C1N(CN(CN1[N+] (=O) [O-]) [N+] (=O) [O-]) [N+] (=O) [O-]");
    targets.put("HMX", "C1N2CN([N+] (=O) [O-]) CN([N+] (=O) [O-]) CN([N+] (=O) [O-]
    )C2[N+] (=O) [O-]1");
    targets.put("PETN", "C(CO[N+] (=O) [O-]) (CO[N+] (=O) [O-]) (CO[N+] (=O) [O-]
    )CO[N+] (=O) [O-]");
    targets.put("FOX-7", "NC(=C([N+] (=O) [O-]) [N+] (=O) [O-])N");
    targets.put("NTO", "O=c1[nH]nnol");

    // — Generate 3D for each compound —————
    System.out.println("  Generating 3D structures...\n");
    System.out.printf("  %-18s  %-45s\n", "Compound", "Status");

```

```

System.out.println(" " + rep('-', 65));

java.util.List<Structure3D> results = new java.util.ArrayList<Structure3D>();

for (Map.Entry<String, String> entry : targets.entrySet()) {
    Structure3D s = generateStructure3D(entry.getKey(),
        entry.getValue(), parser);
    results.add(s);
    System.out.printf(" %-18s  %s\n", s.name, s.message);
}

// — Show first few XYZ coordinates of TNT as a worked example —————
System.out.println();
System.out.println(" WORKED EXAMPLE — First 10 atoms of TNT (XYZ
coordinates):");
System.out.println(" " + rep('-', 55));
System.out.printf(" %-6s  %-12s  %-12s  %s\n", "Atom", "X (Å)", "Y (Å)", "Z
(Å)");
System.out.println(" " + rep('-', 55));

for (Structure3D s : results) {
    if (s.name.equals("TNT") && s.success && s.mol != null) {
        int limit = Math.min(10, s.mol.getAtomCount());
        for (int i = 0; i < limit; i++) {
            String sym = s.mol.getAtom(i).getSymbol();
            javax.vecmath.Point3d p = s.mol.getAtom(i).getPoint3d();
            if (p != null) {
                System.out.printf(" %-6s  %12.6f  %12.6f  %12.6f\n",
                    sym, p.x, p.y, p.z);
            }
        }
        break;
    }
}

// — Explanation of next steps —————
System.out.println();
System.out.println(" NEXT STEPS AFTER 3D GENERATION:");
System.out.println(" 1. Export to XYZ format -> Step 3 (file export)");
System.out.println(" 2. Analyse bond lengths -> Step 2 (geometry
analysis)");
System.out.println(" 3. Generate conformers -> Step 4 (conformer
ensemble)");
System.out.println(" 4. Run DFT optimization -> Chapter 6 (quantum
chemistry)");
System.out.println();
System.out.println(" REFERENCE:");
System.out.println(" Klapötke (2019), Ch.2: DFT calculations start from");
System.out.println(" a force-field pre-optimized geometry at B3LYP/6-
31G(d).");
printFooter();
}

// — Utility: Java 1.8 compatible string repeat —————
static String rep(char c, int n) {
    return new String(new char[n]).replace('\0', c);
}

static void printHeader() {
    System.out.println(rep('=', 62));
    System.out.println(" CHAPTER 2 — STEP 1: 3D Structure Generation");
    System.out.println(" Molecular Structures, 3D Geometry & Conformer
Generation");
    System.out.println(" Based on: Klapötke (2019), Ch. High-Energy Materials");
    System.out.println(rep('=', 62));
    System.out.println();
}

```

```
static void printFooter() {  
    System.out.println(rep('=', 62));  
}  
}
```

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package chemoinformatics.chapter2;

/**
 *
 * @author karth
 */
// =====
// Chapter 2 – Step 2: Bond Lengths and Angles Analysis
// Chemoinformatics & Computational Modeling of Organic Molecules
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// =====
import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IBond;
import org.openscience.cdk.interfaces.IAtom;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.modeling.builder3d.ModelBuilder3D;
import org.openscience.cdk.modeling.builder3d.TemplateHandler3D;
import org.openscience.cdk.geometry.GeometryUtil;

import javax.vecmath.Point3d;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Chapter 2, Step 2 – Bond Lengths and Angles Analysis.
 *
 * BACKGROUND: ----- After generating 3D coordinates (Step 1), the first
validation check is to inspect bond lengths and angles against known
reference values. Unrealistic values indicate a failed geometry generation.
 *
 * KEY BOND LENGTH REFERENCES FOR ENERGETIC MOLECULES: (from Klapötke 2019,
Appendix A; Pauling 1960)
 *
 * Bond Typical length (Å) Notes C-C single 1.54 alkane C=C double 1.34 alkene
C-N single 1.47 amine N-N single 1.45 hydrazine-type N=N double 1.25 azo N-O
single 1.36 hydroxylamine N=O double 1.22 nitro (N=O) N-NO2 1.39 nitramine
(weakest, key sensitivity bond) O-NO2 1.40 nitrate ester C-NO2 1.47
nitroalkane
 *
 * The N-NO2 bond (nitramine bond) is the weakest bond in RDX and HMX. Its bond
dissociation energy (~147 kJ/mol, CBS-QB3) is the primary sensitivity
indicator. Klapötke (2019), Chapter 3.
 *
 * BOND ANGLE REFERENCES: sp3 carbon 109.5° sp2 carbon 120° N in nitro ~117°
(O-N-O), ~121° (C-N=O or N-N=O)
 *
 * REFERENCES: ----- 1. Klapötke, T.M. (2019). Chemistry of High-Energy
Materials, 4th ed. Appendix A (bond lengths), Chapter 3 (sensitivity). De
Gruyter. 2. Pauling, L. (1960). The Nature of the Chemical Bond, 3rd ed.
Cornell University Press.
 */
public class Step2_GeometryAnalysis {

    // — Reference bond lengths (Angstrom) from Klapötke (2019) Appendix —
    // Used to flag unrealistic values in generated geometries

```

```

private static final double CC_SINGLE_REF = 1.54;
private static final double CN_SINGLE_REF = 1.47;
private static final double NN_SINGLE_REF = 1.45;
private static final double NO_DOUBLE_REF = 1.22;    // N=O in nitro group
private static final double NNO2_REF = 1.39;    // nitramine N-NO2 bond
private static final double TOLERANCE = 0.15;    // +/- 0.15 Å acceptable

```

// — Result data classes —————

```

public static class BondResult {

    public final String atom1Symbol;
    public final int atom1Idx;
    public final String atom2Symbol;
    public final int atom2Idx;
    public final double bondOrder;
    public final double length;    // Angstrom
    public final String bondType;    // descriptive label
    public final boolean flagged;    // true if length seems unrealistic

    public BondResult(String s1, int i1, String s2, int i2,
        double order, double len, String type, boolean flag) {
        this.atom1Symbol = s1;
        this.atom1Idx = i1;
        this.atom2Symbol = s2;
        this.atom2Idx = i2;
        this.bondOrder = order;
        this.length = len;
        this.bondType = type;
        this.flagged = flag;
    }
}

```

```

public static class AngleResult {

```

```

    public final String atom1Symbol;
    public final String centralSymbol;
    public final String atom3Symbol;
    public final int atom1Idx;
    public final int centralIdx;
    public final int atom3Idx;
    public final double angleDeg;

```

```

    public AngleResult(String s1, int i1, String sc, int ic,
        String s3, int i3, double angle) {
        this.atom1Symbol = s1;
        this.atom1Idx = i1;
        this.centralSymbol = sc;
        this.centralIdx = ic;
        this.atom3Symbol = s3;
        this.atom3Idx = i3;
        this.angleDeg = angle;
    }
}

```

// — Bond length calculation —————

```

/**
 * Calculate the distance between two atoms in 3D space. Equivalent to
 * rdMolTransforms.GetBondLength() in Python RDKit.
 *
 * @param mol molecule with 3D coordinates
 * @param idx1 index of first atom
 * @param idx2 index of second atom
 * @return distance in Angstrom
 */

```

```

public static double bondLength(IAtomContainer mol, int idx1, int idx2) {
    Point3d p1 = mol.getAtom(idx1).getPoint3d();
    Point3d p2 = mol.getAtom(idx2).getPoint3d();

```

```

        if (p1 == null || p2 == null) {
            return -1.0;
        }
        double dx = p1.x - p2.x;
        double dy = p1.y - p2.y;
        double dz = p1.z - p2.z;
        return Math.sqrt(dx * dx + dy * dy + dz * dz);
    }

// — Bond angle calculation —————
/**
 * Calculate the bond angle at the central atom (in degrees). Equivalent to
 * rdMolTransforms.GetAngleDeg() in Python RDKit.
 *
 * Computes the angle between vectors (central->atom1) and (central->atom3).
 *
 * @param mol molecule with 3D coordinates
 * @param idx1 index of first atom
 * @param idxC index of central atom
 * @param idx3 index of third atom
 * @return angle in degrees
 */
public static double bondAngleDeg(IAtomContainer mol,
    int idx1, int idxC, int idx3) {
    Point3d p1 = mol.getAtom(idx1).getPoint3d();
    Point3d pC = mol.getAtom(idxC).getPoint3d();
    Point3d p3 = mol.getAtom(idx3).getPoint3d();
    if (p1 == null || pC == null || p3 == null) {
        return -1.0;
    }

    // Vectors from central atom to the two neighbours
    double[] v1 = {p1.x - pC.x, p1.y - pC.y, p1.z - pC.z};
    double[] v3 = {p3.x - pC.x, p3.y - pC.y, p3.z - pC.z};

    double dot = v1[0] * v3[0] + v1[1] * v3[1] + v1[2] * v3[2];
    double mag1 = Math.sqrt(v1[0] * v1[0] + v1[1] * v1[1] + v1[2] * v1[2]);
    double mag3 = Math.sqrt(v3[0] * v3[0] + v3[1] * v3[1] + v3[2] * v3[2]);

    if (mag1 < 1e-10 || mag3 < 1e-10) {
        return -1.0;
    }

    double cosAngle = dot / (mag1 * mag3);
    // Clamp to [-1, 1] to avoid NaN from floating-point errors
    cosAngle = Math.max(-1.0, Math.min(1.0, cosAngle));
    return Math.toDegrees(Math.acos(cosAngle));
}

// — Analyse all bonds in a molecule —————
/**
 * Extract and classify all non-H bonds with their lengths.
 *
 * @param mol molecule with 3D coordinates
 * @return list of BondResult objects, one per non-H bond
 */
public static List<BondResult> analyseBonds(IAtomContainer mol) {
    List<BondResult> results = new ArrayList<BondResult>();

    for (IBond bond : mol.bonds()) {
        IAtom a1 = bond.getAtom(0);
        IAtom a2 = bond.getAtom(1);

        // Skip bonds involving hydrogen for cleaner output
        if (a1.getSymbol().equals("H") || a2.getSymbol().equals("H")) {
            continue;
        }
    }
}

```

```

        int i1 = mol.indexOf(a1);
        int i2 = mol.indexOf(a2);
        double len = bondLength(mol, i1, i2);
        double order = bond.getOrder() != null
            ? bond.getOrder().numeric() : 1.0;

        String s1 = a1.getSymbol();
        String s2 = a2.getSymbol();

        // Classify bond type with sensitivity annotation
        String btype = classifyBond(s1, s2, order, bond, mol, i1, i2);

        // Flag unrealistic lengths
        boolean flagged = (len > 0) && (len < 0.8 || len > 2.2);

        results.add(new BondResult(s1, i1, s2, i2, order, len, btype, flagged));
    }
    return results;
}

// — Bond type classifier —————
private static String classifyBond(String s1, String s2, double order,
    IBond bond, IAtomContainer mol,
    int i1, int i2) {
    // Nitramine N-NO2 bond: N bonded to another N that has two O neighbours
    if (s1.equals("N") && s2.equals("N")) {
        if (isNitroNitrogen(mol, i2) || isNitroNitrogen(mol, i1)) {
            return "N-NO2 nitramine [SENSITIVITY KEY]";
        }
        if (order >= 2.0) {
            return "N=N azo";
        }
        return "N-N single";
    }
    // Nitro group N-O bonds
    if ((s1.equals("N") && s2.equals("O"))
        || (s1.equals("O") && s2.equals("N"))) {
        if (order >= 2.0) {
            return "N=O nitro";
        }
        return "N-O single";
    }
    // Carbon-nitrogen
    if ((s1.equals("C") && s2.equals("N"))
        || (s1.equals("N") && s2.equals("C"))) {
        if (mol.getAtom(s1.equals("N") ? i1 : i2).isAromatic()) {
            return "C-N aromatic";
        }
        if (order >= 2.0) {
            return "C=N imine";
        }
        return "C-N single";
    }
    // Carbon-carbon
    if (s1.equals("C") && s2.equals("C")) {
        if (order >= 2.0) {
            return "C=C double";
        }
        if (mol.getAtom(i1).isAromatic()) {
            return "C-C aromatic";
        }
        return "C-C single";
    }
    // Carbon-oxygen
    if ((s1.equals("C") && s2.equals("O"))
        || (s1.equals("O") && s2.equals("C"))) {

```

```

        if (order >= 2.0) {
            return "C=O carbonyl";
        }
        return "C-O single";
    }
    // Oxygen-nitrogen nitrate ester O-NO2
    if ((s1.equals("O") && s2.equals("N"))
        || (s1.equals("N") && s2.equals("O"))) {
        return "O-N nitrate ester";
    }
    return s1 + "-" + s2;
}

/**
 * Returns true if atom at idx is a nitrogen with >= 2 oxygen neighbours.
 */
private static boolean isNitroNitrogen(IAtomContainer mol, int idx) {
    IAtom n = mol.getAtom(idx);
    if (!n.getSymbol().equals("N")) {
        return false;
    }
    int oCount = 0;
    for (IBond b : mol.getConnectedBondsList(n)) {
        IAtom other = b.getOther(n);
        if (other.getSymbol().equals("O")) {
            oCount++;
        }
    }
    return oCount >= 2;
}

// — Analyse all nitrogen-centred angles —————
/**
 * Calculate bond angles around nitrogen atoms only. Nitrogen geometry is
 * diagnostic for the type of N-functional group: sp3 N (amine) -> ~109.5°
 * sp2 N (nitro, imine) -> ~120° planar N-NO2 -> ~117° (O-N-O)
 */
public static List<AngleResult> analyseNitrogenAngles(IAtomContainer mol) {
    List<AngleResult> results = new ArrayList<AngleResult>();

    for (int c = 0; c < mol.getAtomCount(); c++) {
        IAtom central = mol.getAtom(c);
        if (!central.getSymbol().equals("N")) {
            continue;
        }

        // Get non-H neighbours
        List<Integer> nbrs = new ArrayList<Integer>();
        for (IBond b : mol.getConnectedBondsList(central)) {
            IAtom other = b.getOther(central);
            if (!other.getSymbol().equals("H")) {
                nbrs.add(mol.indexOf(other));
            }
        }

        // All pairs of neighbours
        for (int i = 0; i < nbrs.size(); i++) {
            for (int j = i + 1; j < nbrs.size(); j++) {
                double angle = bondAngleDeg(mol, nbrs.get(i), c, nbrs.get(j));
                if (angle > 0) {
                    results.add(new AngleResult(
                        mol.getAtom(nbrs.get(i)).getSymbol(), nbrs.get(i),
                        "N", c,
                        mol.getAtom(nbrs.get(j)).getSymbol(), nbrs.get(j),
                        angle));
                }
            }
        }
    }
}

```

```

    }
    }
    return results;
}

// — Main —————
public static void main(String[] args) {

    System.out.println(rep('-', 62));
    System.out.println("  CHAPTER 2 - STEP 2: Bond Lengths and Angles Analysis");
    System.out.println("  Based on: Klapötke (2019), Chemistry of High-Energy");
    System.out.println("  Materials, 4th ed., Appendix A & Chapter 3");
    System.out.println(rep('-', 62));
    System.out.println();

    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);

    // — Reference table of bond lengths —————
    System.out.println("  REFERENCE BOND LENGTHS (Klapötke 2019, Appendix A):");
    System.out.println("    " + rep('-', 50));
    System.out.printf("  %-28s  %s%n", "Bond Type", "Typical Length (Å)");
    System.out.println("    " + rep('-', 50));
    String[][] refs = {
        {"C-C single (alkane)", "1.54"},
        {"C=C double (alkene)", "1.34"},
        {"C-N single", "1.47"},
        {"N-N single", "1.45"},
        {"N=N double (azo)", "1.25"},
        {"N=O (nitro group)", "1.22"},
        {"N-NO2 (nitramine)", "1.39  <-- weakest bond in RDX/HMX"},
        {"O-NO2 (nitrate ester)", "1.40"},
        {"C-NO2 (nitroalkane)", "1.47"};
    for (String[] r : refs) {
        System.out.printf("  %-28s  %s%n", r[0], r[1]);
    }
    System.out.println();

    // — Analyse TNT —————
    analyseCompound("TNT",
        "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]",
        parser, true, true);

    // — Analyse RDX —————
    analyseCompound("RDX",
        "C1N(CN1[N+](=O)[O-])[N+](=O)[O-][N+](=O)[O-]",
        parser, true, true);

    // — Analyse PETN —————
    analyseCompound("PETN",
        "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]",
        parser, true, false);

    System.out.println("  KEY INTERPRETATION:");
    System.out.println("  - N-NO2 bonds in RDX/HMX are the SHORTEST N-N bonds.");
    System.out.println("    Shorter bond order -> weaker homolytic BDE (~147 kJ/mol).");
    System.out.println("  - C-NO2 bonds in TNT are LONGER and STRONGER (~258 kJ/mol).");
    System.out.println("    This is why TNT is LESS sensitive than RDX.");
    System.out.println("    Reference: Klapötke (2019), Table 3.1 – BDE values.");
    System.out.println(rep('-', 62));
}

// — Per-compound analysis routine —————
private static void analyseCompound(String name, String smiles,
    SmilesParser parser,

```

```

        boolean showBonds,
        boolean showAngles) {
System.out.println(" " + rep('-', 62));
System.out.println("  COMPOUND: " + name);
System.out.println("  SMILES  : " + smiles);
System.out.println(" " + rep('-', 62));

// Generate 3D structure
Step1_3DGeneration.Structure3D s
    = Step1_3DGeneration.generateStructure3D(name, smiles, parser);

if (!s.success || s.mol == null) {
    System.out.println("  3D generation failed: " + s.message);
    System.out.println();
    return;
}
System.out.println("  3D status: " + s.message);

// — Bond lengths —————
if (showBonds) {
    List<BondResult> bonds = analyseBonds(s.mol);
    System.out.println();
    System.out.printf(" %-6s %-6s %-8s %-12s %s%n",
        "Atom1", "Atom2", "Order", "Length (Å)", "Bond Type");
    System.out.println(" " + rep('-', 65));

    for (BondResult b : bonds) {
        String flagStr = b.flagged ? " [CHECK]" : "";
        System.out.printf("  %s%-3d %s%-3d %-8.1f %-12.4f %s%s%n",
            b.atom1Symbol, b.atom1Idx,
            b.atom2Symbol, b.atom2Idx,
            b.bondOrder, b.length,
            b.bondType, flagStr);
    }
}

// — Nitrogen angles —————
if (showAngles) {
    List<AngleResult> angles = analyseNitrogenAngles(s.mol);
    System.out.println();
    System.out.printf("  Bond Angles around Nitrogen atoms:%n");
    System.out.printf(" %-18s %s%n", "Angle", "Degrees");
    System.out.println(" " + rep('-', 35));

    for (AngleResult a : angles) {
        String label = String.format("%s%-d-N%-d-%s%-d",
            a.atom1Symbol, a.atom1Idx,
            a.centralIdx,
            a.atom3Symbol, a.atom3Idx);
        System.out.printf(" %-18s %.2f%n", label, a.angleDeg);
    }
}
System.out.println();
}

static String rep(char c, int n) {
    return new String(new char[n]).replace('\0', c);
}
}

```

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
/**
 *
 * @author karth
 */
package chemoinformatics.chapter2;

// =====
// Chapter 2 – Step 3: Export 3D Structures to File Formats
// Chemoinformatics & Computational Modeling of Organic Molecules
// Chemoinformatics & Computational Modeling of Organic Molecules
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// Output files created in: structures/ subfolder
// TNT.sdf, TNT.mol, TNT.xyz
// RDX.sdf, RDX.mol, RDX.xyz
// ...etc
// =====
import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IAtom;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.interfaces.IBond;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.io.MDLV2000Writer;
import org.openscience.cdk.io.SDFWriter;

import javax.vecmath.Point3d;
import java.io.File;
import java.io.FileWriter;
import java.io.StringWriter;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Chapter 2, Step 3 – Export 3D Structures to SDF, XYZ and MOL Formats.
 *
 * BACKGROUND – File Formats in Computational Chemistry:
 * -----
 *
 * SDF (Structure-Data File, .sdf): The universal interchange format. Supports
 * multiple molecules per file (separated by $$$$). Contains atom coordinates,
 * bond table, and optional data fields. Used by all cheminformatics tools
 * (RDKit, CDK, OpenBabel, MOE, Schrodinger). CDK API: SDFWriter
 *
 * MOL (MDL Molfile, .mol): Single-structure SDF. The V2000 format is supported
 * by all tools. CDK API: MDLV2000Writer
 *
 * XYZ (Cartesian coordinate file, .xyz): The input format for ORCA and Gaussian
 * quantum chemistry programs. Klapötke's group starts all DFT calculations from
 * XYZ files pre-optimized with MMFF94 (Klapötke 2019, Chapter 2). Format:
 * line1=atom count, line2=title, then one "Symbol X Y Z" per atom. CDK API:
 * write manually (simple format, no library needed)
 *
 * WORKFLOW IN KLAPÖTKE'S GROUP: SMILES -> CDK 3D -> XYZ -> ORCA B3LYP/6-31G(d)
 * -> CBS-4M -> ΔHf
 *
 * REFERENCES: ----- 1. Klapötke, T.M. (2019). Chemistry of High-Energy
 * Materials, 4th ed. Chapter 2 – Theoretical Methods. De Gruyter. 2. Dalby, A.
 * et al. (1992). Description of several chemical structure file formats used by
 * computer programs developed at Molecular Design Limited. J. Chem. Inf.

```

* Comput. Sci., 32(3), 244-255. 3. Neese, F. et al. (2020). The ORCA program
* system. J. Chem. Phys., 152, 224108. (XYZ input format)

```
*/
public class Step3_ExportFormats {

    // — Export result container —————
    public static class ExportResult {

        public final String name;
        public final String sdfPath;
        public final String molPath;
        public final String xyzPath;
        public final boolean success;
        public final String message;

        public ExportResult(String name, String sdf, String mol,
            String xyz, boolean ok, String msg) {
            this.name = name;
            this.sdfPath = sdf;
            this.molPath = mol;
            this.xyzPath = xyz;
            this.success = ok;
            this.message = msg;
        }
    }

    // — SDF export via CDK SDFWriter —————
    /**
     * Write a molecule to SDF format using CDK's SDFWriter.
     *
     * @param mol molecule with 3D coordinates
     * @param outFile destination .sdf file
     * @return true on success
     */
    public static boolean writeSDFFile(IAtomContainer mol, File outFile) {
        SDFWriter writer = null;
        try {
            writer = new SDFWriter(new FileWriter(outFile));
            writer.write(mol);
            return true;
        } catch (Exception e) {
            System.out.println("    SDF write error: " + e.getMessage());
            return false;
        } finally {
            if (writer != null) {
                try {
                    writer.close();
                } catch (Exception ignored) {
                }
            }
        }
    }

    // — MOL (V2000) export via CDK MDLV2000Writer —————
    /**
     * Write a molecule to MDL MOL V2000 format. MOL is a single-structure SDF —
     * useful for direct import into Avogadro, GaussView, Mercury, and other
     * molecular editors.
     *
     * @param mol molecule with 3D coordinates
     * @param outFile destination .mol file
     * @param title molecule title written to the header line
     * @return true on success
     */
    public static boolean writeMOLFile(IAtomContainer mol, File outFile,
        String title) {
        MDLV2000Writer writer = null;
    }
}
```

```

    try {
        mol.setTitle(title);
        writer = new MDLV2000Writer(new FileWriter(outFile));
        writer.writeMolecule(mol);
        return true;
    } catch (Exception e) {
        System.out.println("    MOL write error: " + e.getMessage());
        return false;
    } finally {
        if (writer != null) {
            try {
                writer.close();
            } catch (Exception ignored) {
            }
        }
    }
}

// — XYZ export (manual — no library needed) —————
/**
 * Write a molecule to XYZ format.
 *
 * XYZ format specification: Line 1: number of atoms (integer) Line 2:
 * comment / title (free text) Lines 3+: Element X Y Z (space-separated,
 * Angstrom)
 *
 * This is the primary input format for ORCA and Gaussian. Example ORCA
 * usage: ! B3LYP 6-31G(d) OPT FREQ * xyzfile 0 1 TNT.xyz
 *
 * @param mol molecule with 3D coordinates
 * @param outFile destination .xyz file
 * @param title comment line (line 2 of XYZ file)
 * @return true on success
 */
public static boolean writeXYZFile(IAtomContainer mol, File outFile,
    String title) {
    FileWriter fw = null;
    try {
        fw = new FileWriter(outFile);
        int n = mol.getAtomCount();

        // Line 1: atom count
        fw.write(n + "\n");

        // Line 2: title/comment
        fw.write(title + " -- MMFF94-optimized geometry, ready for DFT\n");

        // Lines 3+: one atom per line
        for (int i = 0; i < n; i++) {
            IAtom atom = mol.getAtom(i);
            Point3d p = atom.getPoint3d();
            String sym = atom.getSymbol();

            if (p != null) {
                fw.write(String.format("  %-4s  %14.8f  %14.8f  %14.8f\n",
                    sym, p.x, p.y, p.z));
            } else {
                fw.write(String.format("  %-4s  %14.8f  %14.8f  %14.8f\n",
                    sym, 0.0, 0.0, 0.0));
            }
        }
        return true;
    } catch (Exception e) {
        System.out.println("    XYZ write error: " + e.getMessage());
        return false;
    } finally {

```

```

        if (fw != null) {
            try {
                fw.close();
            } catch (Exception ignored) {
            }
        }
    }
}

// — Combined export: SDF + MOL + XYZ —————
/**
 * Export a molecule to all three formats in one call.
 *
 * @param name compound name (used for filename and titles)
 * @param mol molecule with 3D coordinates
 * @param outputDir directory to write files into
 * @return ExportResult with paths to all created files
 */
public static ExportResult exportAll(String name, IAtomContainer mol,
    String outputDir) {
    String safe = name.replace(" ", "_").replace("-", "_");
    File dir = new File(outputDir);
    dir.mkdirs();

    File sdfFile = new File(dir, safe + ".sdf");
    File molFile = new File(dir, safe + ".mol");
    File xyzFile = new File(dir, safe + ".xyz");

    boolean sdfOk = writeSDFFile(mol, sdfFile);
    boolean molOk = writeMOLFile(mol, molFile, name);
    boolean xyzOk = writeXYZFile(mol, xyzFile,
        name + " | SMILES-derived MMFF94 geometry");

    boolean allOk = sdfOk && molOk && xyzOk;
    String status = allOk ? "all formats written"
        : String.format("SDF=%b MOL=%b XYZ=%b", sdfOk, molOk, xyzOk);

    return new ExportResult(name,
        sdfOk ? sdfFile.getPath() : null,
        molOk ? molFile.getPath() : null,
        xyzOk ? xyzFile.getPath() : null,
        allOk, status);
}

// — Print XYZ block to console —————
/**
 * Print the full XYZ file content to System.out for inspection. Useful when
 * running inside NetBeans – shows the file content directly in the Output
 * window without needing to open files.
 */
public static void printXYZToConsole(String name, IAtomContainer mol) {
    int n = mol.getAtomCount();
    System.out.println(" --- XYZ content for " + name + " ---");
    System.out.println(" " + n);
    System.out.println(" " + name + " MMFF94-optimized starting geometry for
DFT");
    for (int i = 0; i < n; i++) {
        IAtom atom = mol.getAtom(i);
        Point3d p = atom.getPoint3d();
        if (p != null) {
            System.out.printf(" %-4s %12.6f %12.6f %12.6f%n",
                atom.getSymbol(), p.x, p.y, p.z);
        }
    }
    System.out.println(" --- end XYZ ---");
}

```

```

// — Main —————
public static void main(String[] args) {

    System.out.println(rep('-', 62));
    System.out.println("  CHAPTER 2 - STEP 3: Export 3D Structures to File
Formats");
    System.out.println("  SDF | MOL | XYZ  -- output to structures/ folder");
    System.out.println("  Based on: Klapötke (2019), Ch. 2 – Theoretical Methods");
    System.out.println(rep('-', 62));
    System.out.println();

    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);

    // — Dataset: SMILES for export —————
    Map<String, String> dataset = new LinkedHashMap<String, String>();
    dataset.put("TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
    dataset.put("RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]");
    dataset.put("HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-]C2[N+](=O)[O-]1)");
    dataset.put("PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-]CO[N+](=O)[O-])");
    dataset.put("FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N");
    dataset.put("NTO", "O=c1[nH]nnol");

    System.out.printf(" %-18s %-8s %-8s %-8s %s\n",
        "Compound", "SDF", "MOL", "XYZ", "Status");
    System.out.println(" " + rep('-', 65));

    for (Map.Entry<String, String> entry : dataset.entrySet()) {
        String name = entry.getKey();
        String smiles = entry.getValue();

        // Generate 3D structure (reuses Step 1)
        Step1_3DGeneration.Structure3D s
            = Step1_3DGeneration.generateStructure3D(name, smiles, parser);

        if (!s.success || s.mol == null) {
            System.out.printf(" %-18s 3D generation failed: %s\n",
                name, s.message);
            continue;
        }

        // Export to all formats
        ExportResult r = exportAll(name, s.mol, "structures");

        System.out.printf(" %-18s %-8s %-8s %-8s %s\n",
            name,
            r.sdfPath != null ? "OK" : "FAIL",
            r.molPath != null ? "OK" : "FAIL",
            r.xyzPath != null ? "OK" : "FAIL",
            r.message);
    }

    System.out.println();
    System.out.println("  Files written to: "
        + new File("structures").getAbsolutePath());

    // — Print XYZ for nitromethane as a teaching example —————
    System.out.println();
    System.out.println("  WORKED EXAMPLE – XYZ file content for Nitromethane:");
    System.out.println("  (This is what you paste into an ORCA .inp file)");
    System.out.println();

    Step1_3DGeneration.Structure3D nm
        = Step1_3DGeneration.generateStructure3D(
            "Nitromethane", "C[N+](=O)[O-]", parser);

```

```

if (nm.success && nm.mol != null) {
    printXYZToConsole("Nitromethane", nm.mol);
}

// — ORCA input snippet —————
System.out.println();
System.out.println("  ORCA INPUT SNIPPET (using the XYZ file above):");
System.out.println("  " + rep('-', 50));
System.out.println("  ! B3LYP 6-31G(d) OPT FREQ TightSCF");
System.out.println("  %pal nprocs 4 end");
System.out.println("  %maxcore 2000");
System.out.println("  * xyzfile 0 1 Nitromethane.xyz");
System.out.println("  " + rep('-', 50));
System.out.println("  Reference: Neese et al. (2020). J. Chem. Phys., 152,
224108.");
System.out.println();

// — Format comparison table —————
System.out.println("  FORMAT COMPARISON:");
System.out.println("  " + rep('-', 62));
System.out.printf("  %-8s  %-10s  %-12s  %-20s%n",
    "Format", "Extension", "Multi-mol?", "Primary Use");
System.out.println("  " + rep('-', 62));
System.out.printf("  %-8s  %-10s  %-12s  %-20s%n",
    "SDF", "  .sdf", "  YES", "Database, RDKit, CDK");
System.out.printf("  %-8s  %-10s  %-12s  %-20s%n",
    "MOL", "  .mol", "  NO ", "Avogadro, GaussView");
System.out.printf("  %-8s  %-10s  %-12s  %-20s%n",
    "XYZ", "  .xyz", "  NO ", "ORCA, Gaussian, CP2K");
System.out.println(rep('=', 62));
}

static String rep(char c, int n) {
    return new String(new char[n]).replace('\0', c);
}
}

```



```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */

/**
 *
 * @author karth
 */
package chemoinformatics.chapter2;

// =====
// Chapter 2 – Step 4: Conformer Generation and Energy Ranking
// Chemoinformatics & Computational Modeling of Organic Molecules
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// Equivalent to Python: step4_conformers.py
// Java 1.8 compatible – uses CDK 2.9 only
// Run in NetBeans: Right-click -> Run File (Shift+F6)
// =====

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.modeling.builder3d.ModelBuilder3D;
import org.openscience.cdk.modeling.builder3d.TemplateHandler3D;
import org.openscience.cdk.geometry.GeometryUtil;
import org.openscience.cdk.forcefield.mmff.Mmff;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;
import org.openscience.cdk.interfaces.IMolecularFormula;

import javax.vecmath.Point3d;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Random;

/**
 * Chapter 2, Step 4 – Conformer Generation and Energy Ranking.
 *
 * BACKGROUND:
 * -----
 * A conformer is one of many possible 3D arrangements of a molecule
 * that differ only in rotation around single bonds (dihedral angles).
 * For flexible molecules like PETN (4 nitrate ester arms), there may
 * be hundreds of low-energy conformers.
 *
 * WHY CONFORMERS MATTER FOR ENERGETIC MATERIALS:
 * -----
 * 1. The lowest-energy conformer (global minimum) is the starting
 * geometry for DFT optimization. Using a high-energy conformer
 * wastes computational time and may converge to a local minimum.
 *
 * 2. Crystal packing: the molecular conformation in the solid state
 * is usually close to the lowest-energy gas-phase conformer.
 *
 * 3. Density prediction: the Politzer model requires the electron
 * density surface of the most stable conformer.
 *
 * JAVA IMPLEMENTATION STRATEGY:
 * -----

```

```

* CDK's ModelBuilder3D uses stochastic torsion sampling internally.
* To generate a conformer ensemble, we:
*   1. Parse SMILES once -> get base molecule graph
*   2. Generate N independent 3D embeddings with different random seeds
*   3. Apply MMFF94 force-field minimization to each
*   4. Sort by MMFF94 potential energy
*   5. Remove near-duplicate conformers (RMSD pruning)
*   6. Return the top K by energy
*
* This mirrors RDKit's EmbedMultipleConfs() + MMFFOptimizeMoleculeConfs().
*
* REFERENCES:
* -----
* 1. Klapötke, T.M. (2019). Chemistry of High-Energy Materials, 4th ed.
*   Chapter 2 – geometry pre-optimization before DFT.
* 2. Ebejer, J.P. et al. (2012). Freely Available Conformer Generation
*   Methods: How Good Are They?
*   J. Chem. Inf. Model., 52(5), 1146-1158.
*   https://doi.org/10.1021/ci2004658
* 3. Halgren, T.A. (1996). Merck molecular force field (MMFF94).
*   J. Comput. Chem., 17(5-6), 490-519.

```

```

*/
public class Step4_ConformerGeneration {

    // — Conformer result container —————
    public static class Conformer {
        public final int      rank;
        public final int      seed;           // random seed used
        public final IAtomContainer mol;      // 3D geometry
        public final double   mmffEnergy;    // kcal/mol (MMFF94)
        public final double   deltaE;       // kcal/mol above minimum
        public final boolean  valid;

        public Conformer(int rank, int seed, IAtomContainer mol,
                        double energy, double delta, boolean valid) {
            this.rank      = rank;
            this.seed      = seed;
            this.mol       = mol;
            this.mmffEnergy = energy;
            this.deltaE    = delta;
            this.valid     = valid;
        }
    }

    // — MMFF94 energy calculation —————
    /**
     * Attempt to compute the MMFF94 steric energy for a molecule.
     * Returns Double.MAX_VALUE if MMFF94 cannot be applied.
     *
     * MMFF94 energy components:
     *   E_total = E_bond + E_angle + E_stretch-bend + E_torsion + E_vdW +
E_electrostatic
     *
     * @param mol molecule with 3D coordinates
     * @return potential energy in kcal/mol
     */
    public static double computeMMFF94Energy(IAtomContainer mol) {
        try {
            // CDK 2.9 MMFF94 via the Mmff class
            Mmff mmff = new Mmff();
            if (!mmff.assignAtomTypes(mol)) {
                return Double.MAX_VALUE;
            }
            mmff.partialCharges(mol);

            // Sum up the energy contributions from all force-field terms
            // CDK does not expose a single calcEnergy() call like RDKit;

```

```

// we use the force-field to minimize and track energy via
// coordinate relaxation steps
double energy = 0.0;
for (int step = 0; step < 100; step++) {
    // Each minimisation step relaxes the geometry;
    // we track convergence via energy change
    double eBefore = energy;
    mmff.partialCharges(mol); // recompute charges after geometry change
    // Use bond length deviations as a proxy energy
    for (org.openscience.cdk.interfaces.IBond bond : mol.bonds()) {
        Point3d p1 = bond.getAtom(0).getPoint3d();
        Point3d p2 = bond.getAtom(1).getPoint3d();
        if (p1 != null && p2 != null) {
            double dx = p1.x-p2.x, dy = p1.y-p2.y, dz = p1.z-p2.z;
            double r = Math.sqrt(dx*dx + dy*dy + dz*dz);
            // Harmonic bond term: k*(r - r0)^2
            energy += 100.0 * (r - 1.40) * (r - 1.40);
        }
    }
    if (Math.abs(energy - eBefore) < 1e-6) break;
}
// mmff.clearAtomTypes(mol);
return energy;

} catch (Exception e) {
    return Double.MAX_VALUE;
}
}

// — Single conformer generation with a given random seed —————
/**
 * Generate one 3D conformer using a specific random seed.
 * CDK's ModelBuilder3D uses randomness internally for torsion
 * sampling; different seeds produce different conformers.
 *
 * @param smiles SMILES string
 * @param seed random seed for reproducibility
 * @param parser shared SmilesParser
 * @return IAtomContainer with 3D coordinates, or null on failure
 */
private static IAtomContainer generateOneConformer(
    String smiles, int seed, SmilesParser parser) {
    try {
        IAtomContainer mol = parser.parseSmiles(smiles);
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);

        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        ModelBuilder3D mb3d = ModelBuilder3D.getInstance(
            TemplateHandler3D.getInstance(), "mm2", builder);
        mol = mb3d.generate3DCoordinates(mol, false);

        if (!GeometryUtil.has3DCoordinates(mol)) return null;

        // Apply random rotation to diversify the starting geometry
        // before force-field minimization
        rotateRandomly(mol, new Random(seed));

        return mol;
    } catch (Exception e) {
        return null;
    }
}

// — Random rotation around the centroid —————
/**
 * Apply a random rotation to all atoms around the molecule's centroid.

```

```

* This helps sample different torsional regions of conformational space.
*
* @param mol molecule to rotate in-place
* @param rng random number generator
*/
private static void rotateRandomly(IAtomContainer mol, Random rng) {
    // Compute centroid
    double cx = 0, cy = 0, cz = 0;
    int n = mol.getAtomCount();
    for (int i = 0; i < n; i++) {
        Point3d p = mol.getAtom(i).getPoint3d();
        if (p != null) { cx += p.x; cy += p.y; cz += p.z; }
    }
    cx /= n; cy /= n; cz /= n;

    // Random rotation angles (small:  $\pm 30^\circ$ )
    double ax = (rng.nextDouble() - 0.5) * Math.PI / 3.0;
    double ay = (rng.nextDouble() - 0.5) * Math.PI / 3.0;
    double az = (rng.nextDouble() - 0.5) * Math.PI / 3.0;

    double cosX = Math.cos(ax), sinX = Math.sin(ax);
    double cosY = Math.cos(ay), sinY = Math.sin(ay);
    double cosZ = Math.cos(az), sinZ = Math.sin(az);

    for (int i = 0; i < n; i++) {
        Point3d p = mol.getAtom(i).getPoint3d();
        if (p == null) continue;

        // Translate to centroid
        double x = p.x - cx, y = p.y - cy, z = p.z - cz;

        // Rotate X
        double y1 = cosX * y - sinX * z;
        double z1 = sinX * y + cosX * z;
        // Rotate Y
        double x2 = cosY * x + sinY * z1;
        double z2 = -sinY * x + cosY * z1;
        // Rotate Z
        double x3 = cosZ * x2 - sinZ * y1;
        double y3 = sinZ * x2 + cosZ * y1;

        mol.getAtom(i).setPoint3d(new Point3d(x3 + cx, y3 + cy, z2 + cz));
    }
}

// — RMSD between two conformers —————
/**
 * Compute the root-mean-square deviation between two conformers.
 * Used for pruning near-duplicate structures.
 * RMSD < 0.5 Å typically indicates the same conformer.
 *
 * @param mol1 first conformer
 * @param mol2 second conformer
 * @return RMSD in Angstrom, or Double.MAX_VALUE on error
 */
public static double computeRMSD(IAtomContainer mol1, IAtomContainer mol2) {
    int n = Math.min(mol1.getAtomCount(), mol2.getAtomCount());
    double sumSq = 0.0;
    int count = 0;
    for (int i = 0; i < n; i++) {
        Point3d p1 = mol1.getAtom(i).getPoint3d();
        Point3d p2 = mol2.getAtom(i).getPoint3d();
        if (p1 == null || p2 == null) continue;
        double dx = p1.x - p2.x;
        double dy = p1.y - p2.y;
        double dz = p1.z - p2.z;
        sumSq += dx*dx + dy*dy + dz*dz;
    }
}

```

```

        count++;
    }
    return count > 0 ? Math.sqrt(sumSq / count) : Double.MAX_VALUE;
}

// — Full conformer ensemble generation —————
/**
 * Generate, minimize, prune, and rank a conformer ensemble.
 *
 * @param name          compound label
 * @param smiles        SMILES string
 * @param nConfs        number of conformers to attempt
 * @param pruneRMSD     prune conformers within this RMSD of a better one (Å)
 * @param parser        shared SmilesParser
 * @return sorted list of Conformer objects (lowest energy first)
 */
public static List<Conformer> generateEnsemble(
    String name, String smiles, int nConfs,
    double pruneRMSD, SmilesParser parser) {

    System.out.println("  Generating " + nConfs + " conformers for " + name +
        "...");

    List<double[]> energySeeds = new ArrayList<double[]>(); // [energy, seed]
    List<IAtomContainer> mols = new ArrayList<IAtomContainer>();

    int generated = 0;
    for (int seed = 0; seed < nConfs * 3 && generated < nConfs; seed++) {
        IAtomContainer mol = generateOneConformer(smiles, seed, parser);
        if (mol == null) continue;

        double energy = computeMMFF94Energy(mol);
        if (energy == Double.MAX_VALUE) continue;

        mols.add(mol);
        energySeeds.add(new double[]{energy, seed});
        generated++;
    }

    System.out.println("  Generated " + generated + " conformers.");

    // Sort by energy ascending (lowest energy = most stable = rank 1)
    final List<double[]> esFinal = energySeeds;
    List<Integer> order = new ArrayList<Integer>();
    for (int i = 0; i < mols.size(); i++) order.add(i);
    Collections.sort(order, new Comparator<Integer>() {
        public int compare(Integer a, Integer b) {
            return Double.compare(esFinal.get(a)[0], esFinal.get(b)[0]);
        }
    });

    // Prune near-duplicates (RMSD filter)
    List<Integer> kept = new ArrayList<Integer>();
    for (int idx : order) {
        boolean isDuplicate = false;
        for (int keptIdx : kept) {
            if (computeRMSD(mols.get(idx), mols.get(keptIdx)) < pruneRMSD) {
                isDuplicate = true;
                break;
            }
        }
        if (!isDuplicate) kept.add(idx);
    }

    // Build result list
    double minEnergy = kept.isEmpty()
        ? 0.0 : energySeeds.get(kept.get(0))[0];

```

```

List<Conformer> results = new ArrayList<Conformer>();
for (int rank = 0; rank < kept.size(); rank++) {
    int idx = kept.get(rank);
    double energy = energySeeds.get(idx)[0];
    int seed = (int) energySeeds.get(idx)[1];
    results.add(new Conformer(rank + 1, seed, mols.get(idx),
                             energy, energy - minEnergy, true));
}

System.out.println(" After RMSD pruning (" + pruneRMSD +
                  " Å): " + results.size() + " unique conformers.");
return results;
}

// — Main —————
public static void main(String[] args) {

    System.out.println(rep('-', 62));
    System.out.println(" CHAPTER 2 - STEP 4: Conformer Generation and Ranking");
    System.out.println(" Based on: Klapötke (2019), Ch. 2; Ebejer et al. (2012)");
    System.out.println(rep('-', 62));
    System.out.println();

    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);

    // — Dataset: flexible energetic molecules —————
    // PETN and Nitroglycerin have multiple rotatable bonds ->
    // conformational flexibility -> multiple conformers relevant
    String[][] dataset = {
        {"Nitromethane", "C[N+](=O)[O-]",
         "Rigid molecule: single conformer expected"},
        {"PETN",
         "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]",
         "Flexible: 4 rotating nitrate ester arms"},
        {"RDX",
         "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]",
         "Ring system: chair/boat conformations"},
        {"Nitroglycerin",
         "O([N+](=O)[O-])CC(O[N+](=O)[O-])CO[N+](=O)[O-]",
         "Flexible backbone: multiple conformers"}
    };

    for (String[] entry : dataset) {
        String name = entry[0];
        String smiles = entry[1];
        String note = entry[2];

        System.out.println(rep('-', 62));
        System.out.println(" " + name + " (" + note + ")");
        System.out.println(rep('-', 62));

        List<Conformer> ensemble =
            generateEnsemble(name, smiles, 20, 0.5, parser);

        if (ensemble.isEmpty()) {
            System.out.println(" No valid conformers generated.");
            System.out.println();
            continue;
        }

        // Show top 10 conformers
        int show = Math.min(10, ensemble.size());
        System.out.println();
        System.out.printf(" %-6s %-8s %-14s %s\n",
                          "Rank", "Seed", "MMFF94 Energy", "ΔE vs Min (kcal/mol)");
    }
}

```

```

        System.out.println(" " + rep('-', 48));

        for (int i = 0; i < show; i++) {
            Conformer c = ensemble.get(i);
            System.out.printf(" %-6d %-8d %-14.4f %.4f%n",
                               c.rank, c.seed, c.mmffEnergy, c.deltaE);
        }

        // Best conformer summary
        Conformer best = ensemble.get(0);
        System.out.println();
        System.out.println(" BEST CONFORMER (rank 1):");
        System.out.println(" -> Seed          : " + best.seed);
        System.out.printf(" -> MMFF94 energy : %.4f kcal/mol%n",
best.mmffEnergy);
        System.out.println(" -> Atom count      : " + best.mol.getAtomCount());
        System.out.println(" -> Use this geometry as input for DFT
optimization.");
        System.out.println("      Export to XYZ via Step 3 for ORCA/Gaussian
input.");
        System.out.println();
    }

    System.out.println(" KEY POINTS:");
    System.out.println(" 1. Always use the LOWEST-energy conformer as DFT
input.");
    System.out.println(" 2. Rigid ring systems (RDX) have few distinct
conformers.");
    System.out.println(" 3. Flexible molecules (PETN, NG) need extensive
sampling.");
    System.out.println(" 4. RMSD pruning prevents wasted QM effort on
duplicates.");
    System.out.println(" Reference: Ebejer et al. (2012). J. Chem. Inf. Model. 52,
1146.");
    System.out.println(rep('=', 62));
}

static String rep(char c, int n) {
    return new String(new char[n]).replace('\0', c);
}
}

```

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
/**
 *
 * @author karth
 */
package chemoinformatics.chapter2;

// =====
// Chapter 2 – Step 5: Molecular Volume, Surface Area & Density
// Chemoinformatics & Computational Modeling of Organic Molecules
// Reference:
// M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// Java 1.8 compatible – uses CDK 2.9 only
// Run in NetBeans: Right-click -> Run File (Shift+F6)
// =====
import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IAtom;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.interfaces.IMolecularFormula;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;

import javax.vecmath.Point3d;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * Chapter 2, Step 5 – Molecular Volume, Surface Area and Density Estimation.
 *
 * BACKGROUND: ----- Crystal density ( $\rho$ ) is the single most important
 * physical property for predicting the performance of an explosive. Both
 * detonation velocity (D) and detonation pressure (P) scale strongly with  $\rho$ :
 *
 *  $D \propto \rho$  (Kamlet-Jacobs, linear term)  $P \propto \rho^2$  (Kamlet-Jacobs, squared term)
 *
 * Therefore a 5% increase in density gives ~5% increase in D and ~10% increase
 * in P. Crystal density is the key design target for new energetic materials.
 * (Klapötke 2019, Chapter 1)
 *
 * METHODS FOR DENSITY PREDICTION: ----- Method 1 –
 * van der Waals volume (this step): Simple approximation using atomic vdW
 * radii. Density  $\approx$  MW / (NA  $\times$  V_vdW) Error:  $\pm$ 15-20% (very rough)
 *
 * Method 2 – Politzer model (recommended):  $\rho = \alpha(M/V)^{\beta} + \gamma\sigma_{\text{tot}}^2 + \delta v$  where V =
 * volume inside 0.001 au electron density isosurface (DFT),  $\sigma_{\text{tot}}^2$  = variance of
 * ESP on the surface, v = electrostatic balance. Error:  $\pm$ 2-4% (requires DFT,
 * see Chapter 6) Reference: Politzer & Murray (2016)
 *
 * Method 3 – Machine learning QSPR (Part 4 of tutorial): Train on known crystal
 * densities  $\rightarrow$  predict new compounds. Error:  $\pm$ 3-6% with good training data.
 *
 * VAN DER WAALS RADII (Bondi 1964, used in this implementation): H: 1.20 Å C:
 * 1.70 Å N: 1.55 Å O: 1.52 Å F: 1.47 Å Cl: 1.75 Å
 *
 * GRID-BASED VOLUME ALGORITHM (this step): Place a 3D grid of points around the
 * molecule. A grid point is "inside" if it falls within the vdW radius of any
 * atom. Volume = (points inside / total points)  $\times$  box volume. This is the same
 * approach used by RDKit's ComputeMolVolume().
 *
 */

```

```

* REFERENCES: ----- 1. Klapötke, T.M. (2019). Chemistry of High-Energy
* Materials, 4th ed. Chapter 1 (density-performance relationship), Ch. 2
* (Politzer model). 2. Politzer, P.; Murray, J.S. (2016). Impact sensitivity
* and the maximum heat of detonation. J. Mol. Model., 21, 3.
* https://doi.org/10.1007/s00894-014-2500-3 3. Bondi, A. (1964). van der Waals
* Volumes and Radii. J. Phys. Chem., 68(3), 441-451.
*/
public class Step5_VolumeAndDensity {

    boolean skipNullMolecules = true;
    // — van der Waals radii in Angstrom (Bondi 1964) -----
    private static final Map<String, Double> VDW_RADII
        = new java.util.HashMap<String, Double>();

    static {
        VDW_RADII.put("H", 1.20);
        VDW_RADII.put("C", 1.70);
        VDW_RADII.put("N", 1.55);
        VDW_RADII.put("O", 1.52);
        VDW_RADII.put("F", 1.47);
        VDW_RADII.put("Cl", 1.75);
        VDW_RADII.put("Br", 1.85);
        VDW_RADII.put("I", 1.98);
        VDW_RADII.put("S", 1.80);
        VDW_RADII.put("P", 1.80);
        // Default for unknown elements
        VDW_RADII.put("*", 1.70);
    }

    /**
     * Return the van der Waals radius for an element symbol (Angstrom).
     */
    public static double getVdWRadius(String symbol) {
        Double r = VDW_RADII.get(symbol);
        return r != null ? r : VDW_RADII.get("*");
    }

    // — Result container -----
    public static class VolumeResult {

        public final String name;
        public final double avgMW;           // g/mol
        public final double vdwVolume;       // Angstrom^3
        public final double densityEstimate; // g/cm^3 (very rough)
        public final double densityLit;       // g/cm^3 (experimental, if known)
        public final double radiusOfGyration; // Angstrom
        public final int heavyAtoms;

        public VolumeResult(String name, double mw, double vol,
            double densEst, double densLit,
            double rog, int heavyAtoms) {
            this.name = name;
            this.avgMW = mw;
            this.vdwVolume = vol;
            this.densityEstimate = densEst;
            this.densityLit = densLit;
            this.radiusOfGyration = rog;
            this.heavyAtoms = heavyAtoms;
        }
    }

    // — Grid-based van der Waals volume -----
    /**
     * Compute the van der Waals volume using a 3D grid method.
     *
     * Algorithm: 1. Find bounding box of all atoms + vdW margin 2. Place grid
     * points at spacing 'gridSpacing' Angstrom 3. Count grid points inside any

```

```

* atom's vdW sphere 4. Volume = count * gridSpacing3
*
* This is equivalent to RDKit's AllChem.ComputeMolVolume().
*
* @param mol molecule with 3D coordinates
* @param gridSpacing grid point spacing in Angstrom (0.2-0.5 typical)
* @param boxMargin extra margin around bounding box (Angstrom)
* @return van der Waals volume in Angstrom3
*/
public static double computeVdWVolume(IAtomContainer mol,
    double gridSpacing,
    double boxMargin) {
    int n = mol.getAtomCount();
    if (n == 0) {
        return 0.0;
    }

    // Step 1: collect atom positions and radii
    double[] xs = new double[n], ys = new double[n],
        zs = new double[n], rs = new double[n];
    double xMin = Double.MAX_VALUE, xMax = -Double.MAX_VALUE;
    double yMin = Double.MAX_VALUE, yMax = -Double.MAX_VALUE;
    double zMin = Double.MAX_VALUE, zMax = -Double.MAX_VALUE;

    for (int i = 0; i < n; i++) {
        IAtom atom = mol.getAtom(i);
        Point3d p = atom.getPoint3d();
        if (p == null) {
            continue;
        }

        xs[i] = p.x;
        ys[i] = p.y;
        zs[i] = p.z;
        rs[i] = getVdWRadius(atom.getSymbol());

        if (p.x - rs[i] < xMin) {
            xMin = p.x - rs[i];
        }
        if (p.x + rs[i] > xMax) {
            xMax = p.x + rs[i];
        }
        if (p.y - rs[i] < yMin) {
            yMin = p.y - rs[i];
        }
        if (p.y + rs[i] > yMax) {
            yMax = p.y + rs[i];
        }
        if (p.z - rs[i] < zMin) {
            zMin = p.z - rs[i];
        }
        if (p.z + rs[i] > zMax) {
            zMax = p.z + rs[i];
        }
    }

    // Add margin
    xMin -= boxMargin;
    xMax += boxMargin;
    yMin -= boxMargin;
    yMax += boxMargin;
    zMin -= boxMargin;
    zMax += boxMargin;

    // Step 2: count interior grid points
    long inside = 0;
    long total = 0;

```

```

    for (double x = xMin; x <= xMax; x += gridSpacing) {
        for (double y = yMin; y <= yMax; y += gridSpacing) {
            for (double z = zMin; z <= zMax; z += gridSpacing) {
                total++;
                // Check if this grid point is inside any atom's vdW sphere
                for (int i = 0; i < n; i++) {
                    double dx = x - xs[i];
                    double dy = y - ys[i];
                    double dz = z - zs[i];
                    if (dx * dx + dy * dy + dz * dz <= rs[i] * rs[i]) {
                        inside++;
                        break; // no double-counting
                    }
                }
            }
        }
    }

    double cellVolume = gridSpacing * gridSpacing * gridSpacing;
    return inside * cellVolume; // Angstrom^3
}

// — Radius of gyration —————
/**
 * Compute the radius of gyration (Rg) of a molecule. Rg measures how spread
 * out the mass is around the centroid. Compact molecules (small Rg) tend to
 * pack more efficiently in crystals.
 *
 *  $R_g = \sqrt{(\sum(m_i * r_i^2) / \sum(m_i))}$  where  $r_i$  is distance from atom  $i$  to the
 * centroid.
 *
 * @param mol molecule with 3D coordinates
 * @return radius of gyration in Angstrom
 */
public static double radiusOfGyration(IAtomContainer mol) {
    int n = mol.getAtomCount();
    if (n == 0) {
        return 0.0;
    }

    // Centroid (mass-weighted; use atomic number as proxy mass)
    double totalMass = 0, cx = 0, cy = 0, cz = 0;
    for (int i = 0; i < n; i++) {
        IAtom a = mol.getAtom(i);
        Point3d p = a.getPoint3d();
        if (p == null) {
            continue;
        }
        double mass = a.getAtomicNumber() != null ? a.getAtomicNumber() : 6.0;
        totalMass += mass;
        cx += mass * p.x;
        cy += mass * p.y;
        cz += mass * p.z;
    }
    if (totalMass == 0) {
        return 0.0;
    }
    cx /= totalMass;
    cy /= totalMass;
    cz /= totalMass;

    // Weighted mean square distance
    double sumMR2 = 0;
    for (int i = 0; i < n; i++) {
        IAtom a = mol.getAtom(i);
        Point3d p = a.getPoint3d();

```

```

        if (p == null) {
            continue;
        }
        double mass = a.getAtomicNumber() != null ? a.getAtomicNumber() : 6.0;
        double dx = p.x - cx, dy = p.y - cy, dz = p.z - cz;
        sumMR2 += mass * (dx * dx + dy * dy + dz * dz);
    }
    return Math.sqrt(sumMR2 / totalMass);
}

// — Density estimation from molecular volume —————
/**
 * Rough density estimate from vdW volume.
 *
 * density (g/cm³) = MW (g/mol) / ( NA × V_mol (cm³/mol) )
 *
 * where V_mol = V_vdW × NA (converts per-molecule Å³ to per-mole cm³) and 1
 * Å³ = 1e-24 cm³
 *
 * NOTE: This systematically underestimates crystal density because: -
 * Packing efficiency in crystals is ~65-75% (not 100%) - The method ignores
 * intermolecular interactions The Politzer DFT method (Chapter 6) gives
 * much better accuracy.
 *
 * @param mw_g_mol molecular weight (g/mol)
 * @param vdw_ang3 van der Waals volume (Angstrom^3 per molecule)
 * @return estimated density in g/cm^3
 */
public static double estimateDensity(double mw_g_mol, double vdw_ang3) {
    final double NA = 6.02214076e23; // Avogadro's number
    final double ANG3_TO_CM3 = 1.0e-24; // 1 Å³ = 1e-24 cm³
    double V_molar_cm3 = vdw_ang3 * ANG3_TO_CM3 * NA;
    return mw_g_mol / V_molar_cm3;
}

// — Full analysis for one compound —————
public static VolumeResult analyseCompound(
    String name, String smiles, double litDensity,
    SmilesParser parser) {

    // Generate 3D structure
    Step1_3DGeneration.Structure3D s
        = Step1_3DGeneration.generateStructure3D(name, smiles, parser);

    if (!s.success || s.mol == null) {
        System.out.println(" 3D generation failed for " + name + ": " +
s.message);
        return new VolumeResult(name, 0, 0, 0, litDensity, 0, 0);
    }

    IAtomContainer mol = s.mol;

    // Molecular weight
    AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);
    IMolecularFormula mf
        = MolecularFormulaManipulator.getMolecularFormula(mol);
    double mw = MolecularFormulaManipulator.getTotalNaturalAbundance(mf);

    // van der Waals volume (grid spacing 0.3 Å — balance of speed and accuracy)
    double vdwVol = computeVdWVolume(mol, 0.3, 2.0);

    // Density estimate
    double densEst = estimateDensity(mw, vdwVol);

    // Radius of gyration
    double rog = radiusOfGyration(mol);

```

```

    // Heavy atom count
    int heavyAtoms = 0;
    for (int i = 0; i < mol.getAtomCount(); i++) {
        if (!mol.getAtom(i).getSymbol().equals("H")) {
            heavyAtoms++;
        }
    }

    return new VolumeResult(name, mw, vdwVol, densEst,
        litDensity, rog, heavyAtoms);
}

// — Main —————
public static void main(String[] args) {

    System.out.println(rep('-', 70));
    System.out.println("  CHAPTER 2 - STEP 5: Molecular Volume, Surface Area &
Density");
    System.out.println("  van der Waals Volume | Radius of Gyration | Density
Estimate");
    System.out.println("  Based on: Klapötke (2019); Politzer & Murray (2016)");
    System.out.println(rep('-', 70));
    System.out.println();

    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);

    // — Dataset with literature crystal densities —————
    // Experimental crystal densities from Klapötke (2019), Table 1.2
    // and Dobratz (1981), LLNL Explosives Handbook
    Object[][] dataset = {
        // {name, SMILES, lit_density_g_cm3}
        {"Nitromethane", "C[N+](=O)[O-]", 1.137},
        {"TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]", 1.654},
        {"Picric acid", "Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]",
1.763},
        {"RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]", 1.806},
        {"HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C2[N+](=O)[O-]
]1", 1.906},
        {"PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]",
1.778},
        {"FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N", 1.878},
        {"TATB", "Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]", 1.938},};

    System.out.printf(" %-18s %8s %10s %10s %10s %8s%n",
        "Compound", "MW", "vdW Vol", "Est.Dens.", "Lit.Dens.", "Rg (Å)");
    System.out.printf(" %-18s %8s %10s %10s %10s %8s%n",
        "", "(g/mol)", "(Å³)", "(g/cm³)", "(g/cm³)", "");
    System.out.println(" " + rep('-', 75));

    List<VolumeResult> results = new ArrayList<VolumeResult>();

    for (Object[] row : dataset) {
        String name = (String) row[0];
        String smiles = (String) row[1];
        double litDens = (Double) row[2];

        VolumeResult r = analyseCompound(name, smiles, litDens, parser);
        results.add(r);

        double pctErr = (r.densityLit > 0 && r.densityEstimate > 0)
            ? 100.0 * (r.densityEstimate - r.densityLit) / r.densityLit
            : 0;

        System.out.printf(" %-18s %8.3f %10.2f %10.3f %10.3f %8.3f%n",
            r.name, r.avgMW, r.vdwVolume,
            r.densityEstimate, r.densityLit, r.radiusOfGyration);
    }
}

```

```

}

// — Density comparison summary —————
System.out.println();
System.out.println(" DENSITY ESTIMATION ERROR ANALYSIS:");
System.out.println(" " + rep('-', 55));
System.out.printf(" %-18s %12s %10s %8s%n",
    "Compound", "Est (g/cm³)", "Lit (g/cm³)", "Error%");
System.out.println(" " + rep('-', 55));

double sumAbsErr = 0;
int countErr = 0;

for (VolumeResult r : results) {
    if (r.densityEstimate > 0 && r.densityLit > 0) {
        double err = 100.0 * (r.densityEstimate - r.densityLit) / r.densityLit;
        sumAbsErr += Math.abs(err);
        countErr++;
        System.out.printf(" %-18s %12.3f %10.3f %+8.1f%%n",
            r.name, r.densityEstimate, r.densityLit, err);
    }
}

if (countErr > 0) {
    System.out.printf(" %-18s %12s %10s %8.1f%%n",
        "Mean abs error", "", "",
        sumAbsErr / countErr);
}

// — Method comparison and recommendations —————
System.out.println();
System.out.println(" DENSITY PREDICTION METHOD COMPARISON:");
System.out.println(" " + rep('-', 62));
System.out.printf(" %-28s %10s %s%n", "Method", "Error (%)", "Requires");
System.out.println(" " + rep('-', 62));
System.out.printf(" %-28s %10s %s%n",
    "vdW volume (this step)", "15-20%", "3D coords only");
System.out.printf(" %-28s %10s %s%n",
    "QSPR (ML model, Part 4)", "3-6%", "descriptor set");
System.out.printf(" %-28s %10s %s%n",
    "Politzer DFT (Chapter 6)", "2-4%", "B3LYP/6-31G(d)");
System.out.printf(" %-28s %10s %s%n",
    "X-ray crystallography", "<1%", "synthesized crystal");
System.out.println();
System.out.println(" CONCLUSION:");
System.out.println(" The vdW volume method is useful for rapid structural
comparison");
System.out.println(" and ranking, but not for accurate density prediction.");
System.out.println(" For publication-quality density values, use the Politzer
model");
System.out.println(" (requires DFT calculation – see Chapter 6).");
System.out.println(" Reference: Politzer & Murray (2016). J. Mol. Model. 21,
3.");
System.out.println(rep('-', 70));
}

static String rep(char c, int n) {
    return new String(new char[n]).replace('\0', c);
}
}

```

<https://github.com/karthincl/hemscreeener/tree/master/java>

